MODUL 10 · LOCAL STORAGE

Penyimpanan Data Lokal (SharedPreferences & SQLite)

Membangun aplikasi yang datanya tetap ada setelah ditutup. Mempelajari SharedPreferences (SharedPreferencesAsync & WithCache) untuk tema, onboarding, dan sesi login, lalu SQLite dengan sqflite: membuka database, model toMap/fromMap, CRUD, migrasi onUpgrade, query aman dengan whereArgs, transaksi & batch, relasi tabel, hingga studi kasus aplikasi Catatan Kuliah multi-file, ditutup perbandingan dengan Hive dan Supabase.

Durasi Pertemuan 10 (150 Menit) + Belajar Mandiri

Prasyarat Modul 09: Integrasi REST API

Capaian Pembelajaran

- Memilih solusi persistensi lokal yang tepat sesuai jenis dan kompleksitas data.
- Menyimpan dan membaca preferensi (tema, onboarding, sesi login) dengan SharedPreferencesAsync dan SharedPreferencesWithCache.
- Merancang skema tabel SQLite, model toMap/fromMap, dan operasi CRUD dengan sqflite.
- Mengelola perubahan skema dengan version, onCreate, dan onUpgrade tanpa menghilangkan data pengguna.
- Menulis query aman (whereArgs) untuk pencarian, filter, agregasi, serta memakai transaksi, batch, dan relasi foreign key.
- Membangun aplikasi CRUD offline multi-file (Model, Database, Provider, Screen) dan menggunakan path_provider untuk file.

Daftar Isi Modul

1. Mengapa Perlu Penyimpanan Lokal & Pilihannya
2. Dasar SharedPreferences (Async & WithCache)
3. Studi Kasus: Menyimpan Tema Terang/Gelap
4. Studi Kasus: Onboarding & Sesi Login
5. Pengenalan SQLite & Membuka Database
6. Model toMap & fromMap
7. CRUD Lengkap dengan sqflite
8. Migrasi Skema: version, onCreate, onUpgrade
9. Query Lanjutan, rawQuery & SQL Injection
10. Transaksi & Batch
11. Relasi Tabel: Foreign Key & JOIN
12. Studi Kasus: Aplikasi Catatan Kuliah Multi-File
13. Menyimpan File dengan path_provider
14. Perbandingan: SharedPreferences, SQLite, Hive, Supabase
15. Tugas Praktikum
16. Kuis Pemahaman & Kunci Jawaban

1. Mengapa Perlu Penyimpanan Lokal & Pilihannya

Semua variabel di aplikasi Flutter disimpan di **memori (RAM)**. Begitu aplikasi ditutup, isinya hilang. Agar data bertahan (tema pilihan pengguna, status login, catatan kuliah), data harus ditulis ke **penyimpanan lokal** di HP. Proses ini disebut *persistensi*.

Membuktikan Masalahnya

Jalankan contoh berikut, tekan tombol beberapa kali, lalu tutup aplikasi sepenuhnya (swipe dari recent apps) dan buka lagi. Angkanya kembali ke 0.

`lib/main.dart`

```dart
import 'package:flutter/material.dart';

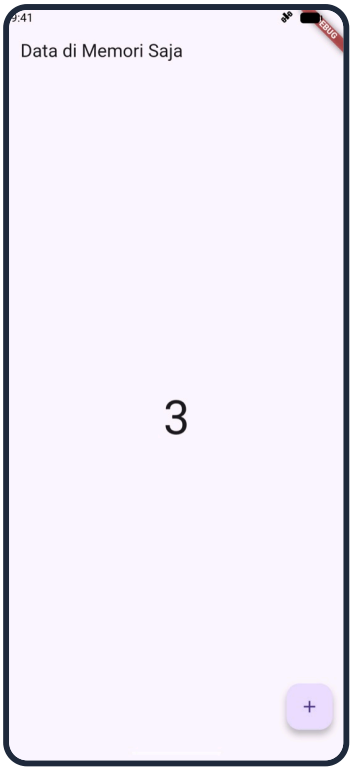
void main() => runApp(const MaterialApp(home: CounterHilang()));

class CounterHilang extends StatefulWidget {
  const CounterHilang({super.key});

  @override
  State<CounterHilang> createState() => _CounterHilangState();
}

class _CounterHilangState extends State<CounterHilang> {
  int _counter = 0; // hanya ada di RAM

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Data di Memori Saja')),
      body: Center(
        child: Text('$_counter', style: Theme.of(context).textTheme.displayLarge),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: () => setState(() => _counter++),
        child: const Icon(Icons.add),
      ),
    );
  }
}
```



Counter di memori: kembali ke 0 setelah aplikasi ditutup

Di section berikutnya, contoh yang sama kita perbaiki dengan SharedPreferences sehingga angka tetap tersimpan.

Pilihan Penyimpanan Lokal di Flutter

Kategori	Package Populer	Struktur Data	Kasus Penggunaan Ideal
Key-Value Store	`shared_preferences`	Key-Value sederhana (String, int, double, bool, List<String>)	Onboarding status, tema gelap/terang, bahasa, sesi login sederhana
Relational Database	`sqflite`	Tabel SQL, kolom, relasi Foreign Key	Data banyak dan terstruktur: catatan, transaksi keuangan, tugas berelasi mata kuliah
NoSQL Fast Store	`hive_ce` (Modul 11)	Box berisi objek Dart	Objek tanpa relasi rumit, baca-tulis sangat cepat tanpa SQL
Penyimpanan Aman	`flutter_secure_storage`	Key-Value terenkripsi (Keychain/Keystore)	Token akses, refresh token, PIN
File	`path_provider` + `dart:io`	File bebas (teks, JSON, gambar)	Ekspor laporan, cache gambar, file unduhan

Cara Memilih

```
Apakah datanya hanya beberapa nilai pengaturan (tema, flag, nama pengguna)?
 ├─ Ya  ──> SharedPreferences
 └─ Tidak
     Apakah datanya rahasia (token, password)?
      ├─ Ya  ──> flutter_secure_storage
      └─ Tidak
          Apakah datanya banyak, perlu dicari/difilter/diurutkan, atau saling berelasi?
           ├─ Ya  ──> SQLite (sqflite)  atau Hive untuk data tanpa relasi
           └─ Tidak, berupa dokumen/gambar ──> File (path_provider)
Perlu dipakai bersama banyak perangkat/pengguna? ──> database cloud (Supabase, Modul 12)
```

⚠️ **Kesalahan Umum**

- Menyimpan ratusan data (mis. daftar transaksi) di SharedPreferences sebagai satu String JSON raksasa. Lambat dan sulit dicari; gunakan database.
- Menganggap penyimpanan lokal sebagai backup. Data lokal hilang saat aplikasi di-uninstall atau "Clear data".
- Menyimpan password asli di penyimpanan lokal. Jangan pernah; simpan token saja dan gunakan penyimpanan aman.

2. Dasar SharedPreferences (Async & WithCache)

`shared_preferences` menyimpan data kecil berbentuk **pasangan kunci-nilai** (key-value), seperti kamus: kunci `'tema'` bernilai `'gelap'`. Di Android data disimpan di DataStore/SharedPreferences, di iOS di NSUserDefaults, dan di web di LocalStorage.

Pasang package: `flutter pub add shared_preferences`

```
flutter pub add shared_preferences
```

Tipe Data yang Didukung

Tipe	Simpan	Baca (null jika belum ada)
`bool`	`setBool('sudahOnboarding', true)`	`getBool('sudahOnboarding')`
`int`	`setInt('counter', 10)`	`getInt('counter')`
`double`	`setDouble('ukuranFont', 1.2)`	`getDouble('ukuranFont')`
`String`	`setString('nama', 'Budi')`	`getString('nama')`
`List<String>`	`setStringList('riwayat', ['a', 'b'])`	`getStringList('riwayat')`

Tiga API: Mana yang Dipakai?

Sejak versi 2.3.0, package ini menyediakan tiga class. Dokumentasi resminya menyatakan class lama `SharedPreferences` akan di-*deprecate* dan menganjurkan dua class baru untuk kode baru.

Class	Cara membuat	Baca data	Kapan dipaka
`SharedPreferencesAsync`	`SharedPreferencesAsync()`	`await` (selalu dari disk)	Pilihan default Selalu memba nilai terbaru
`SharedPreferencesWithCache`	`await SharedPreferencesWithCache.create(...)`	Sinkron (dari cache memori)	Nilai di sangat sering, di `build` atau sa start aplikas
`SharedPreferences` (lama)	`await SharedPreferences.getInstance()`	Sinkron	Kode lama/tu lama. Hindar untuk proyek baru.

Di modul ini kita memakai **`SharedPreferencesAsync`** untuk kebanyakan contoh dan **`SharedPreferencesWithCache`** ketika butuh baca sinkron.

Contoh Kecil: `SharedPreferencesAsync`

```dart
import 'package:shared_preferences/shared_preferences.dart';

Future<void> contohAsync() async {
  final prefs = SharedPreferencesAsync();

  // Simpan
  await prefs.setString('nama', 'Budi');
  await prefs.setInt('semester', 3);
  await prefs.setStringList('matkulFavorit', ['Mobile', 'Basis Data']);

  // Baca: hasilnya nullable, beri nilai default dengan ??
  final nama = await prefs.getString('nama') ?? 'Tamu';
  final semester = await prefs.getInt('semester') ?? 1;
  print('$nama semester $semester');

  // Cek dan hapus
  final ada = await prefs.containsKey('nama');
  await prefs.remove('nama');
```

```
  print('Sebelum dihapus ada: $ada');

  // Hapus beberapa kunci saja (lebih aman daripada clear() tanpa filter)
  await prefs.clear(allowList: {'semester', 'matkulFavorit'});
}
```

Contoh Kecil: `SharedPreferencesWithCache`

Class ini dibuat sekali (async), lalu pembacaan bersifat sinkron tanpa `await`. Parameter `allowList` membatasi kunci yang boleh dipakai; kunci di luar daftar akan memicu error, sehingga typo cepat ketahuan.

```
import 'package:shared_preferences/shared_preferences.dart';

Future<void> contohWithCache() async {
  final prefs = await SharedPreferencesWithCache.create(
    cacheOptions: const SharedPreferencesWithCacheOptions(
      allowList: {'tema', 'counter'},
    ),
  );

  await prefs.setInt('counter', 5); // tulis tetap async
  final int counter = prefs.getInt('counter') ?? 0; // baca sinkron, tanpa await
  print(counter);
}
```

Contoh Lengkap: Counter yang Tidak Hilang

Ini perbaikan dari contoh di section 1. Angka dibaca saat layar dibuka dan disimpan setiap kali bertambah.

`lib/main.dart`

```
// Pastikan sudah: flutter pub add shared_preferences
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

void main() => runApp(const MaterialApp(home: CounterAwet()));

class CounterAwet extends StatefulWidget {
  const CounterAwet({super.key});

  @override
  State<CounterAwet> createState() => _CounterAwetState();
}

class _CounterAwetState extends State<CounterAwet> {
  static const _kunci = 'demo_counter';
  final _prefs = SharedPreferencesAsync();
  int? _counter; // null = sedang membaca dari penyimpanan

  @override
  void initState() {
    super.initState();
    _muat();
  }
```

```dart
  Future<void> _muat() async {
    final nilai = await _prefs.getInt(_kunci) ?? 0;
    if (mounted) setState(() => _counter = nilai);
  }

  Future<void> _tambah() async {
    final baru = (_counter ?? 0) + 1;
    setState(() => _counter = baru); // UI langsung berubah
    await _prefs.setInt(_kunci, baru); // lalu disimpan
  }

  Future<void> _reset() async {
    await _prefs.remove(_kunci);
    if (mounted) setState(() => _counter = 0);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Counter Tersimpan'),
        actions: [IconButton(onPressed: _reset, icon: const Icon(Icons.restart_alt))],
      ),
      body: Center(
        child: _counter == null
            ? const CircularProgressIndicator()
            : Column(
                mainAxisSize: MainAxisSize.min,
                children: [
                  Text('$_counter', style: Theme.of(context).textTheme.displayLarge),
                  const Text('Tutup aplikasi lalu buka lagi, angka tetap ada.'),
                ],
              ),
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _counter == null ? null : _tambah,
        child: const Icon(Icons.add),
      ),
    );
  }
}
```



Counter tersimpan dengan SharedPreferencesAsync

Penjelasan:

- `_counter` bertipe `int?`. Nilai `null` berarti data belum selesai dibaca, sehingga kita menampilkan spinner. Membaca storage selalu butuh waktu.
- Kunci disimpan dalam konstanta `_kunci`. Menulis string `'demo_counter'` berulang kali berisiko typo.
- Pada `_tambah`, UI diperbarui dulu baru disimpan, sehingga aplikasi terasa responsif.

⚠️ **Kesalahan Umum**

- **Menyimpan data penting/besar**. Dokumentasi resmi menegaskan SharedPreferences bukan untuk data kritis. Gunakan untuk pengaturan kecil saja.
- **Lupa** `await` pada `SharedPreferencesAsync`, sehingga yang didapat `Future<int?>`, bukan `int?`.
- Membaca dengan tipe berbeda dari saat menyimpan (simpan `setInt`, baca `getString`): hasilnya error tipe.
- Memanggil `clear()` tanpa `allowList` saat logout. Semua data ikut terhapus, termasuk pengaturan tema.
- Data lama dari `SharedPreferences` versi lama tidak otomatis terbaca oleh `SharedPreferencesAsync` di Android. Untuk migrasi, package menyediakan fungsi `migrateLegacySharedPreferencesToSharedPreferencesAsyncIfNecessary`.

3. Studi Kasus: Menyimpan Tema Terang/Gelap

Kasus nyata pertama: pengguna memilih **tema Terang, Gelap, atau Ikuti Sistem**, dan pilihan itu harus tetap berlaku ketika aplikasi dibuka lagi. Kita gabungkan SharedPreferences dengan Provider (Modul 08).

Konsep

1. `MaterialApp` punya parameter `themeMode` (`ThemeMode.light`, `.dark`, `.system`).
2. Enum tidak bisa disimpan langsung, jadi simpan **namanya** (`ThemeMode.dark.name` = `'dark'`), lalu ubah kembali dengan `ThemeMode.values.byName('dark')`.
3. Tema harus sudah diketahui **sebelum** layar pertama tampil agar tidak "berkedip" dari terang ke gelap. Karena itu preferensi dibaca di `main()` sebelum `runApp` memakai `SharedPreferencesWithCache`.

Contoh Kecil: Enum ke String dan Sebaliknya

```dart
import 'package:flutter/material.dart';

void contohEnum() {
  const mode = ThemeMode.dark;
  final String disimpan = mode.name; // 'dark'
  final ThemeMode dibaca = ThemeMode.values.byName(disimpan); // ThemeMode.dark
  print('$disimpan -> $dibaca');
}
```

Langkah 1: ThemeProvider

`lib/providers/theme_provider.dart`

```dart
import 'package:flutter/material.dart';
import 'package:shared_preferences/shared_preferences.dart';

class ThemeProvider extends ChangeNotifier {
  static const kunciTema = 'theme_mode';

  final SharedPreferencesWithCache _prefs;
  ThemeMode _mode;

  // Nilai awal dibaca sinkron dari cache, jadi tidak perlu loading
  ThemeProvider(this._prefs) : _mode = _baca(_prefs);

  ThemeMode get mode => _mode;

  static ThemeMode _baca(SharedPreferencesWithCache prefs) {
    final nama = prefs.getString(kunciTema);
    if (nama == null) return ThemeMode.system;
    try {
      return ThemeMode.values.byName(nama);
    } on ArgumentError {
      return ThemeMode.system; // nilai rusak/tidak dikenal
    }
```

```
  }

  Future<void> setMode(ThemeMode mode) async {
    if (mode == _mode) return;
    _mode = mode;
    notifyListeners(); // UI berubah seketika
    await _prefs.setString(kunciTema, mode.name); // lalu disimpan
  }
}
```

Langkah 2: Baca Preferensi Sebelum `runApp`

`lib/main.dart`

```
// Pastikan sudah: flutter pub add shared_preferences provider
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'package:shared_preferences/shared_preferences.dart';

import 'providers/theme_provider.dart';

Future<void> main() async {
  // Wajib sebelum memakai plugin di main()
  WidgetsFlutterBinding.ensureInitialized();

  final prefs = await SharedPreferencesWithCache.create(
    cacheOptions: const SharedPreferencesWithCacheOptions(
      allowList: {ThemeProvider.kunciTema},
    ),
  );

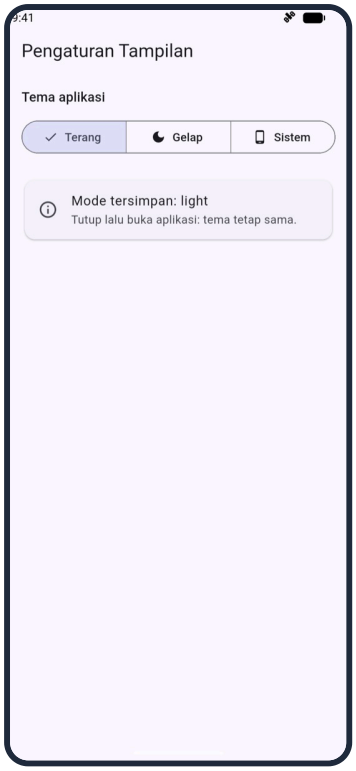
  runApp(
    ChangeNotifierProvider(
      create: (_) => ThemeProvider(prefs),
      child: const TemaApp(),
    ),
  );
}

class TemaApp extends StatelessWidget {
  const TemaApp({super.key});

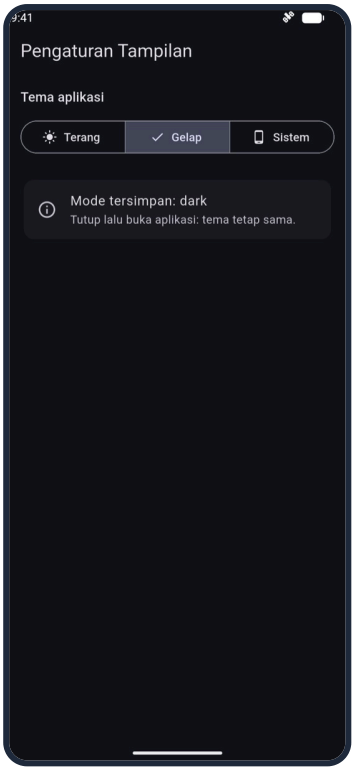
  @override
  Widget build(BuildContext context) {
    final mode = context.watch<ThemeProvider>().mode;
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(colorSchemeSeed: Colors.indigo, brightness: Brightness.light),
      darkTheme: ThemeData(colorSchemeSeed: Colors.indigo, brightness: Brightness.dark),
      themeMode: mode,
      home: const PengaturanScreen(),
    );
  }
}

class PengaturanScreen extends StatelessWidget {
  const PengaturanScreen({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    final themeProv = context.watch<ThemeProvider>();
    return Scaffold(
      appBar: AppBar(title: const Text('Pengaturan Tampilan')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text('Tema aplikasi', style: Theme.of(context).textTheme.titleMedium),
          const SizedBox(height: 12),
          SegmentedButton<ThemeMode>(
            segments: const [
              ButtonSegment(value: ThemeMode.light, icon: Icon(Icons.light_mode), label:
Text('Terang')),
              ButtonSegment(value: ThemeMode.dark, icon: Icon(Icons.dark_mode), label:
Text('Gelap')),
              ButtonSegment(value: ThemeMode.system, icon: Icon(Icons.phone_android), label:
Text('Sistem')),
            ],
            selected: {themeProv.mode},
            onSelectionChanged: (pilihan) => themeProv.setMode(pilihan.first),
          ),
          const SizedBox(height: 24),
          Card(
            child: ListTile(
              leading: const Icon(Icons.info_outline),
              title: Text('Mode tersimpan: ${themeProv.mode.name}'),
              subtitle: const Text('Tutup lalu buka aplikasi: tema tetap sama.'),
            ),
          ),
        ],
      ),
    );
  }
}
```



Mode Terang dipilih



Mode Gelap dipilih dan tersimpan

Penjelasan:

- `WidgetsFlutterBinding.ensureInitialized()` wajib dipanggil karena kita memakai plugin (`SharedPreferencesWithCache.create`) sebelum `runApp`.
- Objek `prefs` dioper ke `ThemeProvider` lewat constructor. Provider tidak perlu membuat instance sendiri dan mudah diganti saat testing.
- `SegmentedButton` adalah komponen Material 3 untuk memilih satu dari beberapa opsi.

> ⚠️ **Kesalahan Umum**
>
> - Lupa `WidgetsFlutterBinding.ensureInitialized()`: muncul error *Binding has not yet been initialized*.
> - Membaca tema di `initState` layar pertama: tampilan sempat terang sebentar lalu berubah gelap (*flicker*).
> - Menyimpan `mode.toString()` (`'ThemeMode.dark'`) lalu membacanya dengan `byName`: gagal. Pakai `.name`.
> - Lupa menambahkan kunci ke `allowList`: `SharedPreferencesWithCache` melempar error saat kunci dipakai.

4. Studi Kasus: Onboarding & Sesi Login

Dua kebutuhan klasik aplikasi: **layar onboarding** yang hanya tampil sekali saat pertama dipasang, dan **sesi login** agar pengguna tidak perlu login setiap membuka aplikasi. Keduanya cukup memakai beberapa nilai `bool` dan `String` di SharedPreferences.

Alur Aplikasi

```
Buka aplikasi
   └─> SplashGate membaca penyimpanan
         ├─ sudahOnboarding == false ──> OnboardingScreen ──(Mulai)──> simpan true ──> Login
         ├─ token == null            ──> LoginScreen ──(Login)──> simpan token ──> Home
         └─ token ada                ──> HomeScreen ──(Logout)──> hapus token ──> Login
```

Langkah 1: SessionManager

Semua akses SharedPreferences dikumpulkan di satu class. Layar tidak perlu tahu nama kunci.

`lib/services/session_manager.dart`

```dart
// lib/services/session_manager.dart
import 'package:shared_preferences/shared_preferences.dart';

class SessionManager {
  static const String keyOnboarding = 'ONBOARDING_DONE';
  static const String keyToken = 'USER_AUTH_TOKEN';
  static const String keyNama = 'USER_NAME';

  final SharedPreferencesAsync _prefs = SharedPreferencesAsync();

  // ---------- Onboarding ----------
  Future<bool> sudahOnboarding() async => await _prefs.getBool(keyOnboarding) ?? false;

  Future<void> selesaiOnboarding() => _prefs.setBool(keyOnboarding, true);

  // ---------- Sesi login ----------
  Future<void> simpanSesiLogin({required String token, required String nama}) async {
    await _prefs.setString(keyToken, token);
    await _prefs.setString(keyNama, nama);
  }

  Future<String?> ambilToken() => _prefs.getString(keyToken);

  Future<String?> ambilNama() => _prefs.getString(keyNama);

  Future<bool> cekStatusLogin() async => await ambilToken() != null;

  // Hapus HANYA data sesi; flag onboarding dan pengaturan lain tetap ada
  Future<void> logout() => _prefs.clear(allowList: {keyToken, keyNama});
}
```

Penjelasan: status login tidak disimpan sebagai `bool` terpisah, melainkan disimpulkan dari ada/tidaknya token. Dengan begitu tidak mungkin terjadi keadaan "login = true tetapi token

kosong". Method `logout()` memakai `clear(allowList: ...)` agar hanya kunci sesi yang terhapus.

> ℹ️ **Tentang Keamanan Token**
>
> Contoh ini cocok untuk belajar. SharedPreferences tidak terenkripsi, jadi untuk aplikasi produksi simpan token di `flutter_secure_storage`. Polanya sama: ganti isi `SessionManager` saja, layar tidak berubah.

Langkah 2: Contoh Lengkap Onboarding + Login

📄 **`lib/main.dart`**

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah: flutter pub add shared_preferences, serta membuat
lib/services/session_manager.dart
import 'package:flutter/material.dart';

import 'services/session_manager.dart';

final session = SessionManager();

void main() {
  runApp(const MaterialApp(debugShowCheckedModeBanner: false, home: SplashGate()));
}

/// Membaca penyimpanan lalu memutuskan layar pertama.
class SplashGate extends StatefulWidget {
  const SplashGate({super.key});

  @override
  State<SplashGate> createState() => _SplashGateState();
}

class _SplashGateState extends State<SplashGate> {
  @override
  void initState() {
    super.initState();
    _tentukanTujuan();
  }

  Future<void> _tentukanTujuan() async {
    final onboarding = await session.sudahOnboarding();
    final login = await session.cekStatusLogin();
    if (!mounted) return;

    final Widget tujuan = !onboarding
        ? const OnboardingScreen()
        : login
            ? const HomeScreen()
            : const LoginScreen();
    // pushReplacement: SplashGate dibuang agar tombol Back tidak kembali ke sini
```

```
    Navigator.pushReplacement(context, MaterialPageRoute(builder: (_) => tujuan));
  }

  @override
  Widget build(BuildContext context) {
    return const Scaffold(body: Center(child: CircularProgressIndicator()));
  }
}

class OnboardingScreen extends StatelessWidget {
  const OnboardingScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: SafeArea(
        child: Padding(
          padding: const EdgeInsets.all(32),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              const Icon(Icons.school, size: 96, color: Colors.indigo),
              const SizedBox(height: 24),
              Text('Selamat Datang di SIA Mobile',
                  style: Theme.of(context).textTheme.headlineSmall, textAlign: TextAlign.center),
              const SizedBox(height: 12),
              const Text('Layar ini hanya muncul sekali saat aplikasi pertama dibuka.',
                  textAlign: TextAlign.center),
              const SizedBox(height: 32),
              FilledButton(
                onPressed: () async {
                  await session.selesaiOnboarding();
                  if (!context.mounted) return;
                  Navigator.pushReplacement(
                    context,
                    MaterialPageRoute(builder: (_) => const LoginScreen()),
                  );
                },
                child: const Text('Mulai'),
              ),
            ],
          ),
        ),
      ),
    );
  }
}

class LoginScreen extends StatefulWidget {
  const LoginScreen({super.key});

  @override
  State<LoginScreen> createState() => _LoginScreenState();
}

class _LoginScreenState extends State<LoginScreen> {
  final _namaCtrl = TextEditingController();

  @override
```

```dart
  void dispose() {
    _namaCtrl.dispose();
    super.dispose();
  }

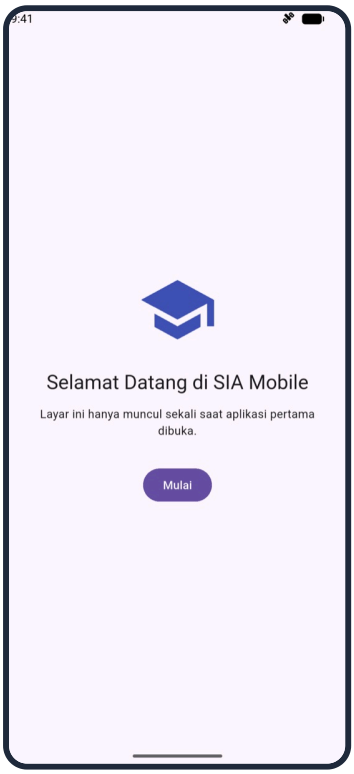
  Future<void> _login() async {
    final nama = _namaCtrl.text.trim();
    if (nama.isEmpty) return;
    // Pada aplikasi nyata token didapat dari API login (Modul 09)
    await session.simpanSesiLogin(token: 'token-${DateTime.now().millisecondsSinceEpoch}', nama:
nama);
    if (!mounted) return;
    Navigator.pushReplacement(context, MaterialPageRoute(builder: (_) => const HomeScreen()));
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Login')),
      body: Padding(
        padding: const EdgeInsets.all(24),
        child: Column(
          children: [
            TextField(
              controller: _namaCtrl,
              decoration: const InputDecoration(labelText: 'Nama', border: OutlineInputBorder()),
            ),
            const SizedBox(height: 16),
            SizedBox(
              width: double.infinity,
              child: FilledButton(onPressed: _login, child: const Text('Masuk')),
            ),
          ],
        ),
      ),
    );
  }
}

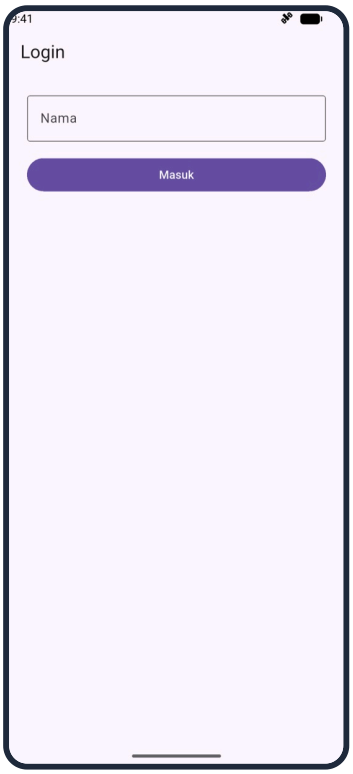
class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Beranda'),
        actions: [
          IconButton(
            tooltip: 'Logout',
            icon: const Icon(Icons.logout),
            onPressed: () async {
              await session.logout();
              if (!context.mounted) return;
              Navigator.pushReplacement(
                context,
                MaterialPageRoute(builder: (_) => const LoginScreen()),
              );
            },
```

```
          ),
        ],
      ),
      body: Center(
        child: FutureBuilder<String?>(
          future: session.ambilNama(),
          builder: (context, snapshot) => Text(
            'Halo, ${snapshot.data ?? '...'}!\nTutup lalu buka aplikasi: kamu tetap login.',
            textAlign: TextAlign.center,
            style: Theme.of(context).textTheme.titleMedium,
          ),
        ),
      ),
    );
  }
}
```



Pembukaan pertama: onboarding



Setelah Mulai: layar login



Setelah login: beranda dengan nama tersimpan

Cara menguji: (1) buka pertama kali, muncul onboarding; (2) tekan Mulai lalu login; (3) tutup aplikasi sepenuhnya dan buka lagi, langsung masuk Beranda; (4) Logout lalu buka lagi, muncul Login (bukan onboarding).

⚠️ **Kesalahan Umum**

- Memakai `Navigator.push` (bukan `pushReplacement`) dari splash/login, sehingga tombol Back kembali ke layar login.
- Logout dengan `clear()` tanpa filter, sehingga onboarding muncul lagi.
- Memakai `context` setelah `await` tanpa cek `mounted` / `context.mounted`.
- Untuk mengulang dari awal saat testing, hapus data aplikasi (Settings → Apps → Clear storage) atau uninstall. Hot restart **tidak** menghapus data tersimpan.

5. Pengenalan SQLite & Membuka Database

Untuk data yang banyak dan terstruktur (daftar catatan, nilai mahasiswa, transaksi), kita memakai **SQLite**: database relasional kecil yang tersimpan sebagai **satu file** di HP. Package `sqflite` menghubungkan Flutter dengan SQLite bawaan Android dan iOS.

```
flutter pub add sqflite path
```

Package `path` dipakai untuk menggabungkan folder dan nama file database dengan benar (`join`).

> ⚠️ **sqflite Tidak Berjalan di Chrome (Web)**
>
> `sqflite` hanya mendukung Android, iOS, dan macOS. Jika dijalankan dengan `flutter run -d chrome`, muncul `MissingPluginException` dan layar kosong. Gunakan emulator Android atau HP fisik.

Kilat SQL: 5 Perintah Dasar

Jika sudah mengambil mata kuliah Basis Data, ini hanya pengingat:

```
-- Membuat tabel
CREATE TABLE mahasiswa (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nim TEXT NOT NULL UNIQUE,
  nama TEXT NOT NULL,
  ipk REAL DEFAULT 0
);

-- Menambah data
INSERT INTO mahasiswa (nim, nama, ipk) VALUES ('H1D022001', 'Budi', 3.5);

-- Membaca data
SELECT * FROM mahasiswa WHERE ipk >= 3.0 ORDER BY nama ASC;

-- Mengubah data
UPDATE mahasiswa SET ipk = 3.8 WHERE id = 1;

-- Menghapus data
DELETE FROM mahasiswa WHERE id = 1;
```

Tipe Data SQLite dan Padanannya di Dart

SQLite	Dart	Catatan
`INTEGER`	`int`	Juga dipakai untuk `bool` (0/1)
`REAL`	`double`	IPK, harga, koordinat
`TEXT`	`String`	Juga untuk `DateTime` (format ISO 8601)
`BLOB`	`Uint8List`	Data biner (jarang dipakai, simpan path file saja)
`NULL`	`null`	Kolom tanpa `NOT NULL` boleh kosong

SQLite **tidak punya** tipe `BOOLEAN` dan `DATETIME` asli. Konversinya kita lakukan sendiri di model (section berikutnya).

Contoh Kecil: Membuka Database

```dart
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

Future<Database> bukaDatabase() async {
  final folder = await getDatabasesPath(); // folder khusus database milik aplikasi
  final path = join(folder, 'kampus.db');   // .../databases/kampus.db

  return openDatabase(
    path,
    version: 1,
    // onCreate hanya dipanggil SEKALI: saat file database belum ada
    onCreate: (db, version) async {
      await db.execute('''
        CREATE TABLE mahasiswa (
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          nim TEXT NOT NULL UNIQUE,
          nama TEXT NOT NULL,
          ipk REAL DEFAULT 0
        )
      ''');
    },
  );
}
```

- `getDatabasesPath()` mengembalikan folder database aplikasi (di Android: `/data/data/<package>/databases`).
- `version` adalah versi **skema** (struktur tabel), bukan versi aplikasi. Dipakai untuk migrasi di section 8.

- `onCreate` berjalan hanya ketika file database belum ada. Mengubah isi `onCreate` tidak berpengaruh pada database yang sudah terbuat.

Pola Singleton: Satu Koneksi untuk Seluruh Aplikasi

Membuka database itu mahal. Simpan koneksinya dan gunakan ulang:

lib/data/app_database.dart

```dart
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class AppDatabase {
  AppDatabase._(); // constructor privat: tidak bisa di-new dari luar
  static final AppDatabase instance = AppDatabase._();

  Database? _db;

  Future<Database> get database async {
    // ??= : buka hanya jika belum pernah dibuka
    return _db ??= await _open();
  }

  Future<Database> _open() async {
    final path = join(await getDatabasesPath(), 'intro_kampus.db');
    return openDatabase(
      path,
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''
          CREATE TABLE mahasiswa (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            nim TEXT NOT NULL UNIQUE,
            nama TEXT NOT NULL,
            ipk REAL DEFAULT 0
          )
        ''');
      },
    );
  }

  Future<void> close() async {
    await _db?.close();
    _db = null;
  }
}
```

Contoh Lengkap: Insert dan Hitung Baris

lib/main.dart

```dart
// Pastikan sudah: flutter pub add sqflite path, dan membuat lib/data/app_database.dart
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart';

import 'data/app_database.dart';
```

```dart
void main() => runApp(const MaterialApp(home: IntroSqliteScreen()));

class IntroSqliteScreen extends StatefulWidget {
  const IntroSqliteScreen({super.key});

  @override
  State<IntroSqliteScreen> createState() => _IntroSqliteScreenState();
}

class _IntroSqliteScreenState extends State<IntroSqliteScreen> {
  int _jumlah = 0;
  String _path = '';
  List<Map<String, Object?>> _baris = [];

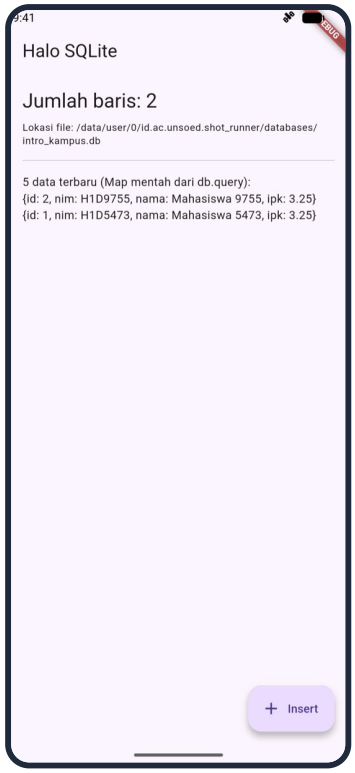
  @override
  void initState() {
    super.initState();
    _refresh();
  }

  Future<void> _refresh() async {
    final db = await AppDatabase.instance.database;
    final jumlah = Sqflite.firstIntValue(await db.rawQuery('SELECT COUNT(*) FROM mahasiswa')) ?? 0;
    final baris = await db.query('mahasiswa', orderBy: 'id DESC', limit: 5);
    if (!mounted) return;
    setState(() {
      _jumlah = jumlah;
      _path = db.path;
      _baris = baris;
    });
  }

  Future<void> _tambah() async {
    final db = await AppDatabase.instance.database;
    final n = DateTime.now().millisecondsSinceEpoch % 100000;
    await db.insert('mahasiswa', {'nim': 'H1D$n', 'nama': 'Mahasiswa $n', 'ipk': 3.25});
    await _refresh();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Halo SQLite')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          Text('Jumlah baris: $_jumlah', style: Theme.of(context).textTheme.headlineSmall),
          const SizedBox(height: 8),
          Text('Lokasi file: $_path', style: Theme.of(context).textTheme.bodySmall),
          const Divider(height: 32),
          const Text('5 data terbaru (Map mentah dari db.query):'),
          for (final row in _baris) Text(row.toString()),
        ],
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: _tambah,
        icon: const Icon(Icons.add),
        label: const Text('Insert'),
      ),
```

```
    );
  }
}
```



Setelah dua kali Insert: jumlah baris dan Map hasil query

Penjelasan: setiap baris hasil `db.query` berbentuk `Map<String, Object?>` dengan nama kolom sebagai kunci. `Sqflite.firstIntValue` mengambil angka pertama dari hasil query seperti `COUNT(*)`. Tutup lalu buka aplikasi: jumlah baris tetap.

⚠ Kesalahan Umum

- Membuka database baru (`openDatabase`) di setiap fungsi. Gunakan singleton.
- Salah ketik SQL di `onCreate` (misalnya koma di akhir kolom terakhir). Error baru muncul saat aplikasi dijalankan, jadi baca pesan `DatabaseException` dengan teliti.
- Mengubah `CREATE TABLE` lalu heran kolom baru tidak ada. `onCreate` tidak dijalankan ulang; lihat section Migrasi.
- Menjalankan di Chrome: sqflite tidak mendukung web.

6. Model toMap & fromMap

Seperti di Modul 09 (JSON), data dari database sebaiknya tidak dipakai sebagai `Map` mentah. Kita buat **model** dengan dua method: `toMap()` untuk menyimpan ke tabel dan `fromMap()` untuk membaca dari tabel.

Konsep: Konversi Tipe

Field Dart	Kolom SQLite	`toMap()`	`fromMap()`
`int? id`	`INTEGER PRIMARY KEY`	tidak ditulis jika null (dibuat otomatis)	`map['id'] as int?`
`bool isSelesai`	`INTEGER`	`isSelesai ? 1 : 0`	`map['isSelesai'] == 1`
`DateTime tenggat`	`TEXT`	`tenggat.toIso8601String()`	`DateTime.parse(...)`
`double ipk`	`REAL`	langsung	`(map['ipk'] as num).toDouble()`

Contoh Kecil

```dart
class Matkul {
  final int? id;
  final String nama;
  final int sks;

  const Matkul({this.id, required this.nama, required this.sks});

  Map<String, Object?> toMap() => {
        if (id != null) 'id': id,
        'nama': nama,
        'sks': sks,
      };

  factory Matkul.fromMap(Map<String, Object?> map) => Matkul(
        id: map['id'] as int?,
        nama: map['nama'] as String,
        sks: map['sks'] as int,
      );
}
```

Model Lengkap: Tugas Kuliah

Model ini dipakai di section CRUD berikutnya. Nama kunci Map harus **sama persis** dengan nama kolom tabel.

lib/models/tugas.dart

```dart
class Tugas {
  final int? id;
  final String judul;
  final String matkul;
  final DateTime tenggat;
  final bool isSelesai;

  const Tugas({
    this.id,
    required this.judul,
    required this.matkul,
    required this.tenggat,
    this.isSelesai = false,
  });

  /// Dart -> baris tabel
  Map<String, Object?> toMap() {
    return {
      if (id != null) 'id': id,
      'judul': judul,
      'matkul': matkul,
      'tenggat': tenggat.toIso8601String(), // DateTime -> TEXT
      'isSelesai': isSelesai ? 1 : 0, // bool -> INTEGER
    };
  }

  /// Baris tabel -> Dart
  factory Tugas.fromMap(Map<String, Object?> map) {
    return Tugas(
      id: map['id'] as int?,
      judul: map['judul'] as String,
      matkul: map['matkul'] as String,
      tenggat: DateTime.parse(map['tenggat'] as String),
      isSelesai: map['isSelesai'] == 1,
    );
  }

  Tugas copyWith({int? id, String? judul, String? matkul, DateTime? tenggat, bool? isSelesai}) {
    return Tugas(
      id: id ?? this.id,
      judul: judul ?? this.judul,
      matkul: matkul ?? this.matkul,
      tenggat: tenggat ?? this.tenggat,
      isSelesai: isSelesai ?? this.isSelesai,
    );
  }

  /// Sisa hari menuju tenggat (negatif berarti terlambat)
  int get sisaHari => tenggat.difference(DateTime.now()).inDays;

  @override
  String toString() => 'Tugas(id: $id, judul: $judul, selesai: $isSelesai)';
}
```

Mencoba Konversi Tanpa Database

`lib/contoh_tugas.dart`

```dart
// Jalankan dengan: dart run lib/contoh_tugas.dart
import 'models/tugas.dart';

void main() {
  final tugas = Tugas(
    judul: 'Laporan Praktikum 10',
    matkul: 'Pemrograman Mobile',
    tenggat: DateTime(2026, 11, 20, 23, 59),
  );

  final map = tugas.toMap();
  print(map);
  // {judul: Laporan Praktikum 10, matkul: Pemrograman Mobile,
  //  tenggat: 2026-11-20T23:59:00.000, isSelesai: 0}

  // Seolah-olah baris ini dibaca dari database (sudah punya id)
  final dariDb = Tugas.fromMap({...map, 'id': 7, 'isSelesai': 1});
  print(dariDb); // Tugas(id: 7, judul: Laporan Praktikum 10, selesai: true)
  print(dariDb.tenggat.year); // 2026
}
```

Penjelasan: `{...map, 'id': 7}` adalah *spread operator*: menyalin isi `map` lalu menambah/menimpa kunci tertentu. Perhatikan `toMap()` tidak menulis `id` ketika null, karena kolom `AUTOINCREMENT` akan mengisinya sendiri.

⚠ Kesalahan Umum

- Menyimpan `bool` langsung (`'isSelesai': true`). sqflite akan menolak atau menyimpan dengan tidak konsisten; selalu ubah ke 0/1.
- Menyimpan `DateTime` langsung: error *Unsupported value*. Ubah ke String ISO 8601 (atau `millisecondsSinceEpoch` sebagai INTEGER).
- Menulis `'id': null` di `toMap()` lalu memakai `ConflictAlgorithm.replace`; lebih aman tidak menulis id sama sekali saat insert.
- Nama kunci di Map berbeda dengan nama kolom (`is_selesai` vs `isSelesai`): muncul error *no such column*.

7. CRUD Lengkap dengan sqflite

Sekarang kita membuat operasi **CRUD** (Create, Read, Update, Delete) lengkap. sqflite menyediakan method siap pakai sehingga kita jarang menulis SQL mentah: `insert`, `query`, `update`, dan `delete`.

Peta Method sqflite

Operasi	Method	Nilai kembali	SQL yang dihasilkan
Create	`db.insert('tugas', map)`	`int` id baris baru	`INSERT INTO tugas (...) VALUES (...)`
Read	`db.query('tugas', where: ..., orderBy: ...)`	`List<Map>`	`SELECT * FROM tugas WHERE ...`
Update	`db.update('tugas', map, where: 'id = ?', whereArgs: [id])`	`int` jumlah baris berubah	`UPDATE tugas SET ... WHERE id = ?`
Delete	`db.delete('tugas', where: 'id = ?', whereArgs: [id])`	`int` jumlah baris terhapus	`DELETE FROM tugas WHERE id = ?`

Langkah 1: DatabaseHelper (Singleton + Tabel + CRUD Map)

Buat folder `lib/data/` lalu file `database_helper.dart`. Versi ini masih bekerja dengan `Map` agar alurnya mudah dilihat.

`lib/data/database_helper.dart`

```dart
// lib/data/database_helper.dart
import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

class DatabaseHelper {
  static final DatabaseHelper instance = DatabaseHelper._init();
  static Database? _database;

  DatabaseHelper._init();

  Future<Database> get database async {
    if (_database != null) return _database!;
    _database = await _initDB('tugas_kampus.db');
    return _database!;
  }

  Future<Database> _initDB(String filePath) async {
    final dbPath = await getDatabasesPath();
    final path = join(dbPath, filePath);
```

```
    return await openDatabase(
      path,
      version: 1,
      onCreate: (db, version) async {
        await db.execute('''
          CREATE TABLE tugas (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            judul TEXT NOT NULL,
            matkul TEXT NOT NULL,
            tenggat TEXT NOT NULL,
            isSelesai INTEGER NOT NULL
          )
        ''');
      },
    );
  }

  // CREATE (Insert)
  Future<int> tambahTugas(Map<String, dynamic> row) async {
    final db = await instance.database;
    return await db.insert('tugas', row);
  }

  // READ (Query All)
  Future<List<Map<String, dynamic>>> ambilSemuaTugas() async {
    final db = await instance.database;
    return await db.query('tugas', orderBy: 'id DESC');
  }

  // UPDATE
  Future<int> updateStatus(int id, bool isSelesai) async {
    final db = await instance.database;
    return await db.update(
      'tugas',
      {'isSelesai': isSelesai ? 1 : 0},
      where: 'id = ?',
      whereArgs: [id],
    );
  }

  // DELETE
  Future<int> hapusTugas(int id) async {
    final db = await instance.database;
    return await db.delete('tugas', where: 'id = ?', whereArgs: [id]);
  }
}
```

Penjelasan: `where: 'id = ?'` berisi tanda tanya sebagai tempat nilai, dan nilainya diberikan lewat `whereArgs`. Jangan pernah menyambung nilai langsung ke string SQL (lihat section Query Lanjutan tentang SQL injection). Tanpa `where`, `update` dan `delete` akan mengenai **semua baris**.

Langkah 2: DAO Bertipe dengan Model `Tugas`

DAO (*Data Access Object*) membungkus helper agar UI bekerja dengan objek `Tugas` (section 6), bukan Map.

`lib/data/tugas_dao.dart`

```dart
import 'package:sqflite/sqflite.dart';

import '../models/tugas.dart';
import 'database_helper.dart';

class TugasDao {
  Future<Database> get _db => DatabaseHelper.instance.database;

  // CREATE: kembalikan objek lengkap dengan id dari database
  Future<Tugas> insert(Tugas tugas) async {
    final db = await _db;
    final id = await db.insert('tugas', tugas.toMap());
    return tugas.copyWith(id: id);
  }

  // READ semua: belum selesai di atas, lalu tenggat terdekat
  Future<List<Tugas>> getAll() async {
    final db = await _db;
    final rows = await db.query('tugas', orderBy: 'isSelesai ASC, tenggat ASC');
    return rows.map(Tugas.fromMap).toList();
  }

  // READ satu
  Future<Tugas?> getById(int id) async {
    final db = await _db;
    final rows = await db.query('tugas', where: 'id = ?', whereArgs: [id], limit: 1);
    return rows.isEmpty ? null : Tugas.fromMap(rows.first);
  }

  // UPDATE seluruh kolom
  Future<int> update(Tugas tugas) async {
    final db = await _db;
    return db.update('tugas', tugas.toMap(), where: 'id = ?', whereArgs: [tugas.id]);
  }

  // DELETE
  Future<int> delete(int id) async {
    final db = await _db;
    return db.delete('tugas', where: 'id = ?', whereArgs: [id]);
  }
}
```

`rows.map(Tugas.fromMap)` adalah *tear-off*: mengoper constructor sebagai fungsi, sama dengan `rows.map((r) => Tugas.fromMap(r))`.

Langkah 3: Contoh Lengkap Aplikasi Tugas Kuliah

Fitur: tambah dan edit lewat dialog, centang selesai (UPDATE), geser untuk hapus (DELETE), dan data tetap ada setelah aplikasi ditutup.

`lib/main.dart`

```dart
// lib/main.dart — salin SELURUH kode ini untuk menggantikan isi lib/main.dart
// Pastikan sudah: flutter pub add sqflite path, serta membuat
```

```dart
// lib/models/tugas.dart, lib/data/database_helper.dart, lib/data/tugas_dao.dart
// CATATAN: sqflite hanya berjalan di Android, iOS, dan macOS. TIDAK bisa di Chrome (web).
import 'package:flutter/material.dart';

import 'data/tugas_dao.dart';
import 'models/tugas.dart';

void main() {
  runApp(MaterialApp(
    debugShowCheckedModeBanner: false,
    theme: ThemeData(colorSchemeSeed: Colors.deepPurple),
    home: const DaftarTugasScreen(),
  ));
}

class DaftarTugasScreen extends StatefulWidget {
  const DaftarTugasScreen({super.key});

  @override
  State<DaftarTugasScreen> createState() => _DaftarTugasScreenState();
}

class _DaftarTugasScreenState extends State<DaftarTugasScreen> {
  final _dao = TugasDao();
  List<Tugas> _daftar = [];
  bool _loading = true;

  @override
  void initState() {
    super.initState();
    _muat();
  }

  Future<void> _muat() async {
    final data = await _dao.getAll();
    if (!mounted) return;
    setState(() {
      _daftar = data;
      _loading = false;
    });
  }

  /// Buka dialog tambah (tugas == null) atau edit (tugas != null)
  Future<void> _bukaForm([Tugas? tugas]) async {
    final hasil = await showDialog<Tugas>(
      context: context,
      builder: (_) => TugasDialog(tugas: tugas),
    );
    if (hasil == null) return; // dibatalkan

    if (hasil.id == null) {
      await _dao.insert(hasil);
    } else {
      await _dao.update(hasil);
    }
    await _muat();
  }

  Future<void> _toggle(Tugas t, bool selesai) async {
```

```
  await _dao.update(t.copyWith(isSelesai: selesai));
  await _muat();
}

Future<void> _hapus(Tugas t) async {
  await _dao.delete(t.id!);
  setState(() => _daftar.remove(t));
  if (!mounted) return;
  ScaffoldMessenger.of(context).showSnackBar(
    SnackBar(
      content: Text('"${t.judul}" dihapus'),
      action: SnackBarAction(
        label: 'URUNGKAN',
        onPressed: () async {
          await _dao.insert(t); // id lama ikut disimpan kembali
          await _muat();
        },
      ),
    ),
  );
}

@override
Widget build(BuildContext context) {
  final selesai = _daftar.where((t) => t.isSelesai).length;
  return Scaffold(
    appBar: AppBar(title: Text('Tugas Kuliah ($selesai/${_daftar.length} selesai)')),
    body: _loading
        ? const Center(child: CircularProgressIndicator())
        : _daftar.isEmpty
            ? const Center(child: Text('Belum ada tugas. Tekan tombol + untuk menambah.'))
            : ListView.builder(
                itemCount: _daftar.length,
                itemBuilder: (context, index) {
                  final t = _daftar[index];
                  final telat = !t.isSelesai && t.sisaHari < 0;
                  return Dismissible(
                    key: ValueKey(t.id),
                    direction: DismissDirection.endToStart,
                    background: Container(
                      color: Colors.red,
                      alignment: Alignment.centerRight,
                      padding: const EdgeInsets.only(right: 20),
                      child: const Icon(Icons.delete, color: Colors.white),
                    ),
                    onDismissed: (_) => _hapus(t),
                    child: CheckboxListTile(
                      value: t.isSelesai,
                      onChanged: (v) => _toggle(t, v ?? false),
                      title: Text(
                        t.judul,
                        style: TextStyle(
                          decoration: t.isSelesai ? TextDecoration.lineThrough : null,
                        ),
                      ),
                      subtitle: Text(
                        '${t.matkul} • ${telat ? 'TERLAMBAT' : '${t.sisaHari} hari lagi'}',
                        style: TextStyle(color: telat ? Colors.red : null),
                      ),
```

```
                        secondary: IconButton(
                          icon: const Icon(Icons.edit_outlined),
                          onPressed: () => _bukaForm(t),
                        ),
                      ),
                    );
                  },
                ),
      floatingActionButton: FloatingActionButton(
        tooltip: 'Tambah tugas',
        onPressed: () => _bukaForm(),
        child: const Icon(Icons.add),
      ),
    );
  }
}

/// Dialog form. Controller dimiliki State dialog sendiri sehingga
/// di-dispose otomatis SETELAH animasi tutup dialog selesai.
class TugasDialog extends StatefulWidget {
  final Tugas? tugas;
  const TugasDialog({super.key, this.tugas});

  @override
  State<TugasDialog> createState() => _TugasDialogState();
}

class _TugasDialogState extends State<TugasDialog> {
  late final _judulCtrl = TextEditingController(text: widget.tugas?.judul);
  late final _matkulCtrl =
      TextEditingController(text: widget.tugas?.matkul ?? 'Pemrograman Mobile');
  late DateTime _tenggat =
      widget.tugas?.tenggat ?? DateTime.now().add(const Duration(days: 7));

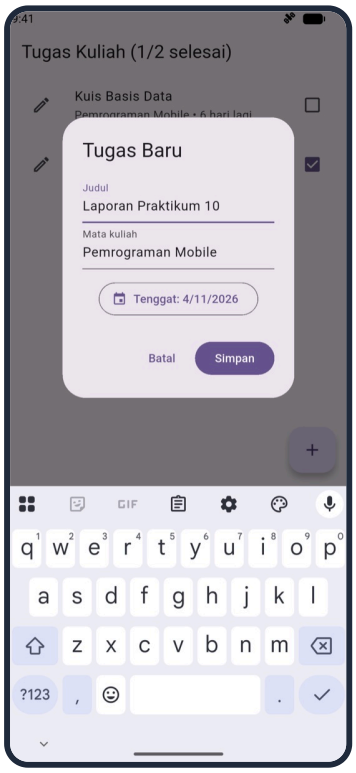
  @override
  void dispose() {
    _judulCtrl.dispose();
    _matkulCtrl.dispose();
    super.dispose();
  }

  Future<void> _pilihTanggal() async {
    final dipilih = await showDatePicker(
      context: context,
      initialDate: _tenggat,
      firstDate: DateTime(2020),
      lastDate: DateTime(2035),
    );
    if (dipilih != null) setState(() => _tenggat = dipilih);
  }

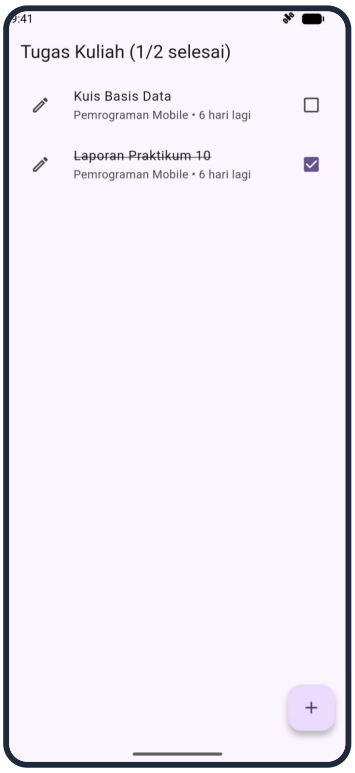
  void _simpan() {
    final judul = _judulCtrl.text.trim();
    if (judul.isEmpty) return;
    final lama = widget.tugas;
    final data = lama == null
        ? Tugas(judul: judul, matkul: _matkulCtrl.text.trim(), tenggat: _tenggat)
        : lama.copyWith(judul: judul, matkul: _matkulCtrl.text.trim(), tenggat: _tenggat);
    Navigator.pop(context, data); // kirim hasil ke pemanggil showDialog
```

```
  }

  @override
  Widget build(BuildContext context) {
    return AlertDialog(
      title: Text(widget.tugas == null ? 'Tugas Baru' : 'Edit Tugas'),
      content: Column(
        mainAxisSize: MainAxisSize.min,
        children: [
          TextField(controller: _judulCtrl, decoration: const InputDecoration(labelText: 'Judul')),
          TextField(controller: _matkulCtrl, decoration: const InputDecoration(labelText: 'Mata
kuliah')),
          const SizedBox(height: 12),
          OutlinedButton.icon(
            icon: const Icon(Icons.event),
            label: Text('Tenggat: ${_tenggat.day}/${_tenggat.month}/${_tenggat.year}'),
            onPressed: _pilihTanggal,
          ),
        ],
      ),
      actions: [
        TextButton(onPressed: () => Navigator.pop(context), child: const Text('Batal')),
        FilledButton(onPressed: _simpan, child: const Text('Simpan')),
      ],
    );
  }
}
```



Dialog tambah tugas



Dua tugas tersimpan, satu sudah dicentang selesai

Penjelasan baris penting:

- Setelah setiap perubahan kita memanggil `_muat()` agar list selalu sama dengan isi database (*single source of truth*).
- Form dibuat sebagai widget terpisah `TugasDialog` yang mengembalikan objek `Tugas` lewat `Navigator.pop(context, data)`. Controller dimiliki dialog sehingga di-dispose setelah dialog benar-benar tertutup.
- Fitur URUNGKAN memasukkan kembali objek lama **beserta id-nya**, sehingga posisinya tetap sama.
- Pada `Dismissible`, item dihapus dari list dulu (`_daftar.remove`) karena Flutter mewajibkan item yang sudah di-dismiss segera hilang dari tree.

> ⚠️ **Kesalahan Umum**
>
> - `db.update('tugas', map)` tanpa `where`: seluruh tugas ikut berubah.
> - Lupa memuat ulang data setelah insert/update sehingga layar tidak berubah walaupun database sudah berubah.
> - Membuat `TextEditingController` di method lalu langsung `dispose()` setelah `showDialog` selesai: dialog masih beranimasi menutup sehingga muncul error *_dependents.isEmpty*. Simpan controller di State widget dialog.
> - Error *A dismissed Dismissible widget is still part of the tree*: item belum dihapus dari list setelah digeser.
> - Error *UNIQUE constraint failed*: menyisipkan id/NIM yang sudah ada. Tangani dengan `try/catch` atau `conflictAlgorithm`.

8. Migrasi Skema: version, onCreate, onUpgrade

Aplikasi terus berkembang. Di versi 2 kamu ingin menambah kolom `prioritas` pada tabel tugas. Masalahnya, pengguna lama sudah punya database versi 1 berisi data penting. Kita tidak boleh menghapusnya; kita harus **memigrasi** struktur tabel. Inilah fungsi `version` dan `onUpgrade`.

Konsep: Kapan Callback Dipanggil?

Kondisi di HP pengguna	Callback yang berjalan
Aplikasi baru dipasang (file belum ada)	`onCreate(db, versiTerbaru)`
File ada, versinya lebih kecil dari `version`	`onUpgrade(db, versiLama, versiBaru)`
File ada, versinya lebih besar (aplikasi di-downgrade)	`onDowngrade` (jika diisi)
Setiap kali database dibuka, sebelum yang lain	`onConfigure(db)`
Setiap kali database selesai dibuka	`onOpen(db)`

Contoh Kecil: Menambah Kolom di Versi 2

```dart
import 'package:sqflite/sqflite.dart';

Future<Database> buka(String path) {
  return openDatabase(
    path,
    version: 2, // naik dari 1 ke 2
    onCreate: (db, version) async {
      // Pengguna BARU langsung mendapat struktur terbaru
      await db.execute('''
        CREATE TABLE tugas (
          id INTEGER PRIMARY KEY AUTOINCREMENT,
          judul TEXT NOT NULL,
          prioritas INTEGER NOT NULL DEFAULT 0
        )
      ''');
    },
    onUpgrade: (db, oldVersion, newVersion) async {
      // Pengguna LAMA (versi 1) cukup ditambah kolomnya, data tetap aman
      if (oldVersion < 2) {
        await db.execute('ALTER TABLE tugas ADD COLUMN prioritas INTEGER NOT NULL DEFAULT 0');
      }
    },
  );
}
```

Aturan emas: `onCreate` **selalu membuat struktur terbaru**, sedangkan `onUpgrade` **berisi langkah perubahan dari setiap versi lama**.

Pola Migrasi Bertahap

Pengguna bisa melompat versi (dari 1 langsung ke 3). Karena itu tulis setiap langkah dengan `if (oldVersion < N)` yang berurutan, **bukan** `if (oldVersion == N)`.

`lib/data/migrasi_database.dart`

```dart
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

/// Riwayat skema:
///  v1: tabel tugas (id, judul)
///  v2: + kolom prioritas
///  v3: + tabel catatan
class MigrasiDatabase {
  static const int versiSkema = 3;
  static Database? _db;

  static Future<Database> get database async => _db ??= await _open();

  static Future<Database> _open() async {
    final path = join(await getDatabasesPath(), 'migrasi_demo.db');
    return openDatabase(
      path,
      version: versiSkema,
      onConfigure: (db) async {
        // Aktifkan foreign key setiap kali dibuka (dipakai di section Relasi)
        await db.execute('PRAGMA foreign_keys = ON');
      },
      onCreate: (db, version) async {
        // Struktur TERBARU untuk pengguna baru
        await db.execute('''
          CREATE TABLE tugas (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            judul TEXT NOT NULL,
            prioritas INTEGER NOT NULL DEFAULT 0
          )
        ''');
        await _buatTabelCatatan(db);
      },
      onUpgrade: (db, oldVersion, newVersion) async {
        if (oldVersion < 2) {
          await db.execute(
            'ALTER TABLE tugas ADD COLUMN prioritas INTEGER NOT NULL DEFAULT 0',
          );
        }
        if (oldVersion < 3) {
          await _buatTabelCatatan(db);
        }
      },
    );
  }

  static Future<void> _buatTabelCatatan(Database db) async {
```

```
    await db.execute('''
      CREATE TABLE catatan (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        isi TEXT NOT NULL,
        dibuat TEXT NOT NULL
      )
    ''');
  }

  /// Daftar kolom suatu tabel, untuk melihat hasil migrasi
  static Future<List<String>> kolom(String tabel) async {
    final db = await database;
    final info = await db.rawQuery('PRAGMA table_info($tabel)');
    return info.map((r) => '${r['name']} (${r['type']})').toList();
  }
}
```

Penjelasan: fungsi `_buatTabelCatatan` dipakai di dua tempat (onCreate dan onUpgrade) sehingga definisi tabel tidak ditulis dua kali. `PRAGMA table_info` adalah perintah SQLite untuk melihat struktur tabel.

Contoh Lengkap: Melihat Struktur Database

lib/main.dart

```
// Pastikan sudah: flutter pub add sqflite path, dan membuat lib/data/migrasi_database.dart
import 'package:flutter/material.dart';
import 'package:sqflite/sqflite.dart'; // untuk db.getVersion()

import 'data/migrasi_database.dart';

void main() => runApp(const MaterialApp(home: StrukturScreen()));

class StrukturScreen extends StatefulWidget {
  const StrukturScreen({super.key});

  @override
  State<StrukturScreen> createState() => _StrukturScreenState();
}

class _StrukturScreenState extends State<StrukturScreen> {
  // Future disimpan sekali agar query tidak diulang setiap rebuild
  late final Future<Map<String, List<String>>> _future = _muat();

  Future<Map<String, List<String>>> _muat() async {
    final db = await MigrasiDatabase.database;
    return {
      'Versi skema': ['${await db.getVersion()}'],
      'Tabel tugas': await MigrasiDatabase.kolom('tugas'),
      'Tabel catatan': await MigrasiDatabase.kolom('catatan'),
    };
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Hasil Migrasi')),
```

```
      body: FutureBuilder(
        future: _future,
        builder: (context, snapshot) {
          if (snapshot.hasError) return Center(child: Text('${snapshot.error}'));
          if (!snapshot.hasData) return const Center(child: CircularProgressIndicator());
          return ListView(
            padding: const EdgeInsets.all(16),
            children: [
              for (final entry in snapshot.data!.entries)
                Card(
                  child: ListTile(
                    title: Text(entry.key),
                    subtitle: Text(entry.value.join('\n')),
                  ),
                ),
            ],
          );
        },
      ),
    );
  }
}
```



Versi skema 3 dengan kolom dan tabel terbaru

Coba eksperimen: ubah `versiSkema` menjadi 1 dan hapus bagian yang belum ada di v1, jalankan (setelah uninstall), lalu kembalikan ke 3 tanpa uninstall. Data lama tetap ada, dan kolom/tabel baru muncul.

💡 **Fase Belajar vs Produksi**

Saat masih belajar dan belum ada pengguna, cara paling cepat setelah mengubah `onCreate` adalah **uninstall aplikasi** (atau Clear storage) lalu jalankan ulang. Setelah aplikasi dirilis, jangan lakukan itu: selalu naikkan `version` dan tulis `onUpgrade`.

⚠️ **Kesalahan Umum**

- Mengubah `CREATE TABLE` tanpa menaikkan `version`: pengguna lama mendapat error *no such column*.
- Hot reload/hot restart tidak membuat ulang database. Perubahan skema baru terlihat setelah migrasi atau uninstall.
- Menulis `if (oldVersion == 1)`: pengguna yang melompat dari v1 ke v3 tidak mendapat langkah v2 → v3.
- Menurunkan nomor `version`: memicu `onDowngrade` atau error. Nomor versi hanya boleh naik.
- SQLite hanya mendukung sebagian `ALTER TABLE` (tambah/ganti nama kolom). Untuk mengubah tipe kolom, buat tabel baru, salin data, hapus tabel lama.

9. Query Lanjutan, rawQuery & SQL Injection

Kekuatan utama SQLite dibanding SharedPreferences adalah kemampuan **mencari, memfilter, mengurutkan, dan menghitung** data. Di section ini kita mengenal parameter lengkap `db.query()`, `rawQuery()` untuk SQL bebas, dan cara aman memasukkan input pengguna ke query.

Parameter `db.query()`

```dart
import 'package:sqflite/sqflite.dart';

Future<List<Map<String, Object?>>> contohQuery(Database db) {
  return db.query(
    'mahasiswa',                        // FROM mahasiswa
    columns: ['nim', 'nama', 'ipk'],    // SELECT nim, nama, ipk  (null = semua kolom)
    where: 'prodi = ? AND ipk >= ?',    // WHERE prodi = ? AND ipk >= ?
    whereArgs: ['Informatika', 3.0],    // nilai untuk tanda ? secara berurutan
    orderBy: 'ipk DESC, nama ASC',      // ORDER BY
    limit: 10,                          // LIMIT 10
    offset: 0,                          // OFFSET 0 (untuk halaman berikutnya: 10, 20, ...)
  );
}
```

⚠️ SQL Injection: Mengapa Wajib Memakai `?` dan `whereArgs`

Bayangkan fitur pencarian yang menyambung input pengguna langsung ke SQL:

```dart
import 'package:sqflite/sqflite.dart';

// ❌ BERBAHAYA: input pengguna disambung langsung ke string SQL
Future<List<Map<String, Object?>>> cariTidakAman(Database db, String input) {
  return db.rawQuery("SELECT * FROM mahasiswa WHERE nama = '$input'");
}

// ✅ AMAN: nilai dikirim terpisah, tidak pernah dianggap perintah SQL
Future<List<Map<String, Object?>>> cariAman(Database db, String input) {
  return db.rawQuery('SELECT * FROM mahasiswa WHERE nama = ?', [input]);
}
```

Jika pengguna mengetik `x' OR '1'='1`, versi tidak aman menjadi `WHERE nama = 'x' OR '1'='1'` yang **selalu benar**, sehingga semua data bocor. Input seperti `'; DROP TABLE mahasiswa; --` bahkan bisa merusak data. Dengan `?`, input tersebut diperlakukan sebagai teks biasa. Tanda kutip di nama seperti `Ma'ruf` pun tidak membuat error.

Pencarian dengan LIKE

```dart
import 'package:sqflite/sqflite.dart';

Future<List<Map<String, Object?>>> cariNama(Database db, String kata) {
  return db.query(
    'mahasiswa',
```

```
    where: 'nama LIKE ? OR nim LIKE ?',
    whereArgs: ['%$kata%', '%$kata%'], // % = karakter apa saja, ditaruh di whereArgs
  );
}
```

Tanda `%` ditempel pada **nilai** di `whereArgs`, bukan di string `where`.

`rawQuery` untuk Agregasi

Untuk `COUNT`, `AVG`, `GROUP BY`, dan `JOIN`, SQL mentah lebih jelas:

```dart
import 'package:sqflite/sqflite.dart';

Future<void> statistik(Database db) async {
  // Satu angka: gunakan Sqflite.firstIntValue
  final total = Sqflite.firstIntValue(
    await db.rawQuery('SELECT COUNT(*) FROM mahasiswa'),
  ) ?? 0;

  // Rata-rata IPK per prodi
  final rows = await db.rawQuery('''
    SELECT prodi, COUNT(*) AS jumlah, ROUND(AVG(ipk), 2) AS rata
    FROM mahasiswa
    GROUP BY prodi
    ORDER BY rata DESC
  ''');
  print('Total: $total');
  for (final r in rows) {
    print('${r['prodi']}: ${r['jumlah']} mhs, rata-rata ${r['rata']}');
  }
}
```

Contoh Lengkap: Pencarian & Filter Mahasiswa

`lib/data/mahasiswa_query_db.dart`

```dart
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class MahasiswaQueryDb {
  static Database? _db;

  static Future<Database> get database async => _db ??= await openDatabase(
        join(await getDatabasesPath(), 'query_demo.db'),
        version: 1,
        onCreate: (db, version) async {
          await db.execute('''
            CREATE TABLE mahasiswa (
              id INTEGER PRIMARY KEY AUTOINCREMENT,
              nim TEXT NOT NULL UNIQUE,
              nama TEXT NOT NULL,
              prodi TEXT NOT NULL,
              ipk REAL NOT NULL
            )
          ''');
          // Data contoh
```

```
          const contoh = [
            ['H1D022001', 'Budi Santoso', 'Informatika', 3.45],
            ['H1D022002', 'Siti Aminah', 'Informatika', 3.82],
            ['H1D022003', "Ma'ruf Hidayat", 'Informatika', 2.95],
            ['H1A022004', 'Dewi Lestari', 'Teknik Elektro', 3.60],
            ['H1A022005', 'Rizky Pratama', 'Teknik Elektro', 3.10],
            ['H1C022006', 'Andi Wijaya', 'Teknik Sipil', 3.25],
          ];
          for (final m in contoh) {
            await db.insert('mahasiswa', {'nim': m[0], 'nama': m[1], 'prodi': m[2], 'ipk': m[3]});
          }
        },
      );

  /// Gabungan pencarian + filter prodi + filter IPK minimum
  static Future<List<Map<String, Object?>>> cari({
    String kata = '',
    String? prodi,
    double ipkMin = 0,
  }) async {
    final db = await database;
    final kondisi = <String>['ipk >= ?'];
    final args = <Object?>[ipkMin];
    if (kata.isNotEmpty) {
      kondisi.add('(nama LIKE ? OR nim LIKE ?)');
      args.addAll(['%$kata%', '%$kata%']);
    }
    if (prodi != null) {
      kondisi.add('prodi = ?');
      args.add(prodi);
    }
    return db.query(
      'mahasiswa',
      where: kondisi.join(' AND '),
      whereArgs: args,
      orderBy: 'ipk DESC',
    );
  }

  static Future<List<String>> daftarProdi() async {
    final db = await database;
    final rows = await db.rawQuery('SELECT DISTINCT prodi FROM mahasiswa ORDER BY prodi');
    return rows.map((r) => r['prodi'] as String).toList();
  }

  static Future<double> rataIpk() async {
    final db = await database;
    final rows = await db.rawQuery('SELECT AVG(ipk) AS rata FROM mahasiswa');
    return (rows.first['rata'] as num? ?? 0).toDouble();
  }
}
```

Penjelasan: fungsi `cari` menyusun klausa `WHERE` secara dinamis. String `where` hanya berisi nama kolom dan `?` yang kita tulis sendiri, sedangkan semua nilai dari pengguna masuk ke `args`. Itulah cara aman membuat filter dinamis.

lib/main.dart

```dart
// Pastikan sudah: flutter pub add sqflite path, dan membuat lib/data/mahasiswa_query_db.dart
import 'package:flutter/material.dart';

import 'data/mahasiswa_query_db.dart';

void main() => runApp(const MaterialApp(home: CariMahasiswaScreen()));

class CariMahasiswaScreen extends StatefulWidget {
  const CariMahasiswaScreen({super.key});

  @override
  State<CariMahasiswaScreen> createState() => _CariMahasiswaScreenState();
}

class _CariMahasiswaScreenState extends State<CariMahasiswaScreen> {
  String _kata = '';
  String? _prodi;
  bool _cumlaude = false;
  List<String> _semuaProdi = [];
  List<Map<String, Object?>> _hasil = [];
  double _rata = 0;

  @override
  void initState() {
    super.initState();
    _init();
  }

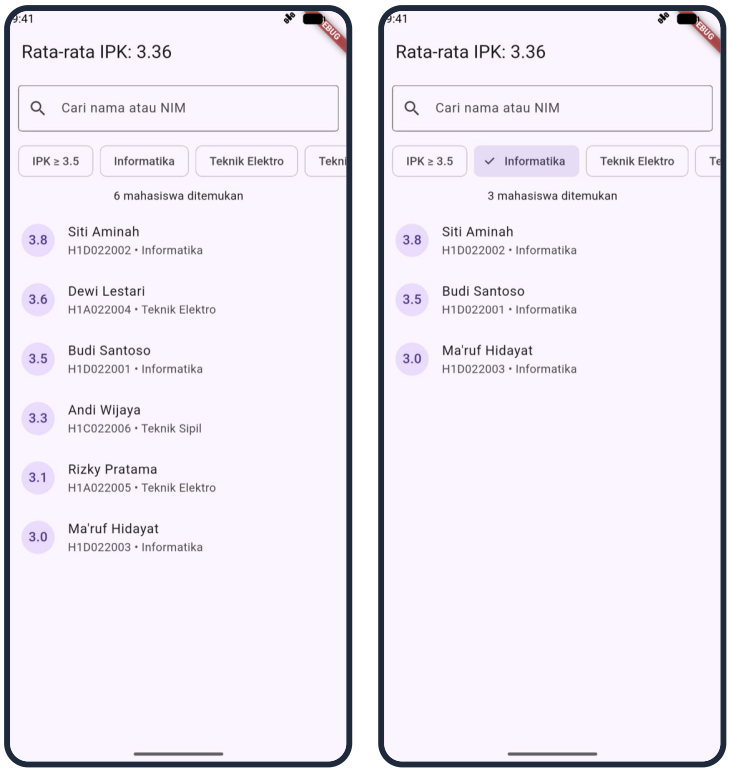
  Future<void> _init() async {
    final prodi = await MahasiswaQueryDb.daftarProdi();
    final rata = await MahasiswaQueryDb.rataIpk();
    if (!mounted) return;
    setState(() {
      _semuaProdi = prodi;
      _rata = rata;
    });
    await _cari();
  }

  Future<void> _cari() async {
    final hasil = await MahasiswaQueryDb.cari(
      kata: _kata,
      prodi: _prodi,
      ipkMin: _cumlaude ? 3.5 : 0,
    );
    if (mounted) setState(() => _hasil = hasil);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Rata-rata IPK: ${_rata.toStringAsFixed(2)}')),
      body: Column(
        children: [
          Padding(
            padding: const EdgeInsets.all(12),
            child: TextField(
```

```
            decoration: const InputDecoration(
              prefixIcon: Icon(Icons.search),
              hintText: 'Cari nama atau NIM',
              border: OutlineInputBorder(),
            ),
            onChanged: (v) {
              _kata = v.trim();
              _cari();
            },
          ),
        ),
        SingleChildScrollView(
          scrollDirection: Axis.horizontal,
          padding: const EdgeInsets.symmetric(horizontal: 12),
          child: Row(
            children: [
              FilterChip(
                label: const Text('IPK ≥ 3.5'),
                selected: _cumlaude,
                onSelected: (v) {
                  setState(() => _cumlaude = v);
                  _cari();
                },
              ),
              for (final p in _semuaProdi)
                Padding(
                  padding: const EdgeInsets.only(left: 8),
                  child: ChoiceChip(
                    label: Text(p),
                    selected: _prodi == p,
                    onSelected: (v) {
                      setState(() => _prodi = v ? p : null);
                      _cari();
                    },
                  ),
                ),
            ],
          ),
        ),
        Padding(
          padding: const EdgeInsets.all(8),
          child: Text('${_hasil.length} mahasiswa ditemukan'),
        ),
        Expanded(
          child: ListView.builder(
            itemCount: _hasil.length,
            itemBuilder: (context, i) {
              final m = _hasil[i];
              return ListTile(
                leading: CircleAvatar(child: Text((m['ipk'] as num).toStringAsFixed(1))),
                title: Text(m['nama'] as String),
                subtitle: Text('${m['nim']} • ${m['prodi']}'),
              );
            },
          ),
        ),
      ],
    ),
  );
```

```
  }
}
```



Tampilan awal diurutkan berdasarkan IPK

Filter IPK di atas 3.5 dan prodi Informatika

Coba ketik `ma'ruf` di kolom pencarian: tetap berjalan normal karena memakai `whereArgs`. Dengan penyambungan string, tanda kutip itu akan membuat SQL error.

> ⚠️ **Kesalahan Umum**
>
> - Menyambung input ke SQL dengan `'$input'` (SQL injection).
> - Jumlah `?` tidak sama dengan jumlah elemen `whereArgs` : error *bind or column index out of range*.
> - Menulis `where: "nama LIKE '%?%'"`. Tanda `?` di dalam kutip tidak dianggap parameter; letakkan `%` di nilai.
> - Mengambil semua data lalu memfilter di Dart dengan `where()` untuk ribuan baris. Biarkan SQLite yang memfilter, jauh lebih cepat.

10. Transaksi & Batch

Beberapa operasi harus berhasil **semua atau tidak sama sekali**. Contoh pengisian KRS: menambah baris KRS *dan* mengurangi kuota kelas. Jika langkah kedua gagal, langkah pertama harus dibatalkan. Inilah **transaksi**. Selain itu, ada **batch** untuk menjalankan banyak perintah sekaligus dengan cepat.

Konsep

	`db.transaction`	`db.batch`
Tujuan	Keutuhan data (atomik)	Kecepatan banyak perintah
Jika ada error	Semua perubahan di dalamnya dibatalkan (rollback)	Jika di-commit tanpa transaksi, perintah sebelumnya bisa tetap tersimpan
Bisa membaca hasil di tengah?	Ya (`await txn.query`)	Tidak, hasil didapat setelah `commit()`
Contoh	Transfer saldo, ambil kuota KRS	Impor 1.000 data awal

Contoh Kecil: Transaksi

```dart
import 'package:sqflite/sqflite.dart';

Future<void> transfer(Database db, int dari, int ke, int jumlah) async {
  await db.transaction((txn) async {
    // PENTING: di dalam transaksi gunakan txn, BUKAN db
    await txn.rawUpdate('UPDATE dompet SET saldo = saldo - ? WHERE id = ?', [jumlah, dari]);
    final rows = await txn.query('dompet', columns: ['saldo'], where: 'id = ?', whereArgs: [dari]);
    if ((rows.first['saldo'] as int) < 0) {
      throw Exception('Saldo tidak cukup'); // melempar = ROLLBACK otomatis
    }
    await txn.rawUpdate('UPDATE dompet SET saldo = saldo + ? WHERE id = ?', [jumlah, ke]);
  });
}
```

Contoh Kecil: Batch

```dart
import 'package:sqflite/sqflite.dart';

Future<void> imporNilai(Database db, List<Map<String, Object?>> data) async {
  await db.transaction((txn) async {
    final batch = txn.batch();
    for (final row in data) {
      batch.insert('nilai', row); // belum dijalankan, hanya dikumpulkan
    }
    await batch.commit(noResult: true); // dijalankan sekaligus
```

```
  });
}
```

Batch di dalam transaksi adalah kombinasi terbaik: cepat **dan** aman. `noResult: true` membuat lebih cepat lagi karena kita tidak butuh id setiap baris.

Contoh Lengkap: Simulasi Pengisian KRS

`lib/data/krs_database.dart`

```dart
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

class KrsDatabase {
  static Database? _db;

  static Future<Database> get database async => _db ??= await openDatabase(
        join(await getDatabasesPath(), 'krs_demo.db'),
        version: 1,
        onCreate: (db, version) async {
          await db.execute('''
            CREATE TABLE kelas (
              id INTEGER PRIMARY KEY,
              nama TEXT NOT NULL,
              kuota INTEGER NOT NULL
            )
          ''');
          await db.execute('''
            CREATE TABLE krs (
              id INTEGER PRIMARY KEY AUTOINCREMENT,
              kelas_id INTEGER NOT NULL,
              waktu TEXT NOT NULL
            )
          ''');
          await db.insert('kelas', {'id': 1, 'nama': 'Pemrograman Mobile A', 'kuota': 2});
          await db.insert('kelas', {'id': 2, 'nama': 'Basis Data B', 'kuota': 5});
        },
      );

  /// Ambil kelas: tambah KRS + kurangi kuota, dalam SATU transaksi.
  static Future<void> ambilKelas(int kelasId) async {
    final db = await database;
    await db.transaction((txn) async {
      await txn.insert('krs', {
        'kelas_id': kelasId,
        'waktu': DateTime.now().toIso8601String(),
      });
      final rows = await txn.query('kelas', where: 'id = ?', whereArgs: [kelasId]);
      final kuota = rows.first['kuota'] as int;
      if (kuota <= 0) {
        // Insert KRS di atas ikut dibatalkan karena exception ini
        throw StateError('Kuota kelas sudah penuh');
      }
      await txn.update('kelas', {'kuota': kuota - 1}, where: 'id = ?', whereArgs: [kelasId]);
    });
  }
```

```
  /// Isi 500 baris KRS dummy memakai batch.
  static Future<Duration> imporDummy() async {
    final db = await database;
    final sw = Stopwatch()..start();
    await db.transaction((txn) async {
      final batch = txn.batch();
      for (var i = 0; i < 500; i++) {
        batch.insert('krs', {'kelas_id': 2, 'waktu': 'dummy-$i'});
      }
      await batch.commit(noResult: true);
    });
    return sw.elapsed;
  }

  static Future<List<Map<String, Object?>>> ringkasan() async {
    final db = await database;
    return db.rawQuery('''
      SELECT k.id, k.nama, k.kuota, COUNT(r.id) AS terisi
      FROM kelas k LEFT JOIN krs r ON r.kelas_id = k.id
      GROUP BY k.id
    ''');
  }
}
```

lib/main.dart

```
// Pastikan sudah: flutter pub add sqflite path, dan membuat lib/data/krs_database.dart
import 'package:flutter/material.dart';

import 'data/krs_database.dart';

void main() => runApp(const MaterialApp(home: KrsScreen()));

class KrsScreen extends StatefulWidget {
  const KrsScreen({super.key});

  @override
  State<KrsScreen> createState() => _KrsScreenState();
}

class _KrsScreenState extends State<KrsScreen> {
  List<Map<String, Object?>> _kelas = [];
  String _log = 'Tekan "Ambil" berkali-kali sampai kuota habis.';

  @override
  void initState() {
    super.initState();
    _muat();
  }

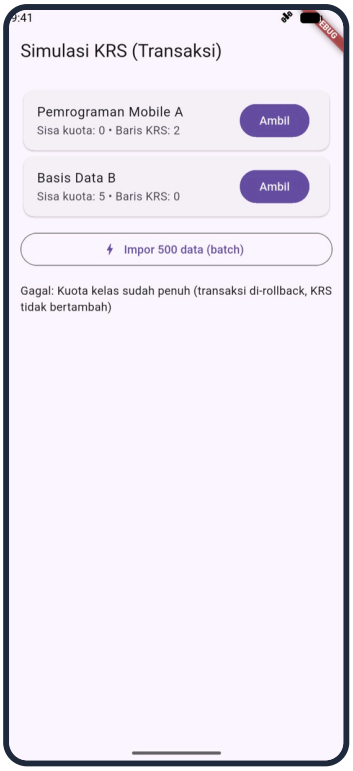
  Future<void> _muat() async {
    final data = await KrsDatabase.ringkasan();
    if (mounted) setState(() => _kelas = data);
  }

  Future<void> _ambil(int id) async {
    try {
      await KrsDatabase.ambilKelas(id);
```

```
      _log = 'Berhasil mengambil kelas $id';
    } on StateError catch (e) {
      _log = 'Gagal: ${e.message} (transaksi di-rollback, KRS tidak bertambah)';
    }
    await _muat();
  }

  Future<void> _impor() async {
    final durasi = await KrsDatabase.imporDummy();
    _log = '500 baris dimasukkan dengan batch dalam ${durasi.inMilliseconds} ms';
    await _muat();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Simulasi KRS (Transaksi)')),
      body: ListView(
        padding: const EdgeInsets.all(16),
        children: [
          for (final k in _kelas)
            Card(
              child: ListTile(
                title: Text(k['nama'] as String),
                subtitle: Text('Sisa kuota: ${k['kuota']} • Baris KRS: ${k['terisi']}'),
                trailing: FilledButton(
                  onPressed: () => _ambil(k['id'] as int),
                  child: const Text('Ambil'),
                ),
              ),
            ),
          const SizedBox(height: 12),
          OutlinedButton.icon(
            onPressed: _impor,
            icon: const Icon(Icons.bolt),
            label: const Text('Impor 500 data (batch)'),
          ),
          const SizedBox(height: 16),
          Text(_log),
        ],
      ),
    );
  }
}
```



Setelah Ambil ditekan 3 kali: kuota 0, baris KRS tetap 2

Penjelasan: pada kelas "Pemrograman Mobile A" (kuota 2), tekan Ambil tiga kali. Pada percobaan ketiga, insert ke tabel `krs` sebenarnya sudah dijalankan, tetapi karena exception dilempar sebelum transaksi selesai, SQLite membatalkan insert tersebut. Angka "Baris KRS" tetap 2.

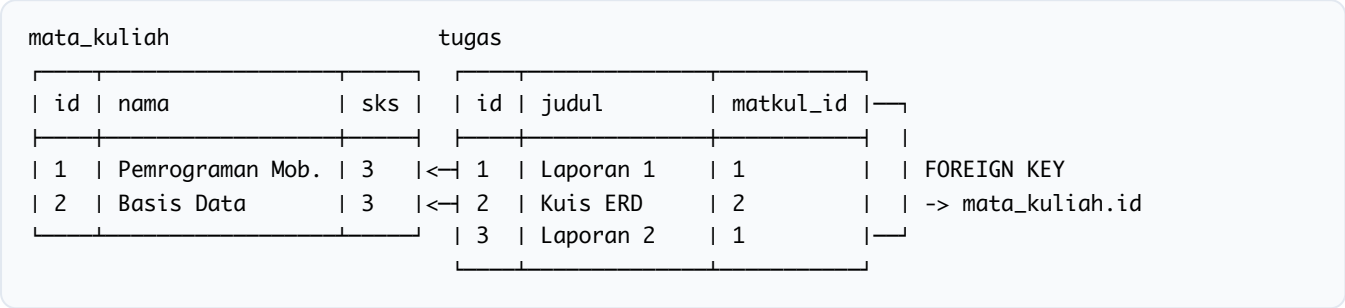
⚠️ Kesalahan Umum

- **Memakai `db` di dalam `db.transaction`**: aplikasi macet (*deadlock*) karena `db` menunggu transaksi selesai. Selalu pakai `txn`.
- Menangkap exception di dalam blok transaksi (try/catch di dalamnya) sehingga rollback tidak terjadi. Biarkan exception keluar dari callback.
- Insert ribuan baris satu per satu dengan `await db.insert` di loop. Gunakan batch.
- Lupa `await batch.commit()`: tidak ada data yang tersimpan.

11. Relasi Tabel: Foreign Key & JOIN

Database relasional bisa menghubungkan tabel. Contoh: satu **mata kuliah** punya banyak **tugas** (relasi *one-to-many*). Daripada menulis nama mata kuliah berulang di setiap tugas, tabel tugas cukup menyimpan `matkul_id` yang menunjuk ke tabel mata kuliah.

Rancangan Tabel



Konsep Penting

- `FOREIGN KEY (matkul_id) REFERENCES mata_kuliah(id)` memastikan tugas hanya bisa menunjuk mata kuliah yang ada.
- `ON DELETE CASCADE` : jika mata kuliah dihapus, semua tugasnya ikut terhapus otomatis.
- SQLite **mematikan** foreign key secara default. Aktifkan dengan `PRAGMA foreign_keys = ON` di `onConfigure` (dijalankan setiap database dibuka).
- `JOIN` menggabungkan baris dari dua tabel dalam satu query.

Contoh Kecil: Skema dan JOIN

```
CREATE TABLE mata_kuliah (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  nama TEXT NOT NULL,
  sks INTEGER NOT NULL
);

CREATE TABLE tugas (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  judul TEXT NOT NULL,
  matkul_id INTEGER NOT NULL,
  FOREIGN KEY (matkul_id) REFERENCES mata_kuliah(id) ON DELETE CASCADE
);

-- Tugas beserta nama mata kuliahnya
SELECT t.id, t.judul, m.nama AS matkul_nama
FROM tugas t
JOIN mata_kuliah m ON m.id = t.matkul_id
ORDER BY m.nama;
```

Model untuk Hasil JOIN

Hasil JOIN berisi kolom dari dua tabel, jadi kita buat model khusus untuk tampilan:

lib/models/relasi_models.dart

```dart
class MataKuliah {
  final int? id;
  final String nama;
  final int sks;
  final int jumlahTugas; // hasil COUNT, bukan kolom tabel

  const MataKuliah({this.id, required this.nama, required this.sks, this.jumlahTugas = 0});

  Map<String, Object?> toMap() => {if (id != null) 'id': id, 'nama': nama, 'sks': sks};

  factory MataKuliah.fromMap(Map<String, Object?> m) => MataKuliah(
        id: m['id'] as int?,
        nama: m['nama'] as String,
        sks: m['sks'] as int,
        jumlahTugas: m['jumlah_tugas'] as int? ?? 0,
      );
}

class TugasDenganMatkul {
  final int id;
  final String judul;
  final String matkulNama;

  const TugasDenganMatkul({required this.id, required this.judul, required this.matkulNama});

  factory TugasDenganMatkul.fromMap(Map<String, Object?> m) => TugasDenganMatkul(
        id: m['id'] as int,
        judul: m['judul'] as String,
        matkulNama: m['matkul_nama'] as String, // alias dari "AS matkul_nama"
      );
}
```

Database Helper dengan Relasi

lib/data/relasi_database.dart

```dart
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

import '../models/relasi_models.dart';

class RelasiDatabase {
  static Database? _db;

  static Future<Database> get database async => _db ??= await openDatabase(
        join(await getDatabasesPath(), 'relasi_demo.db'),
        version: 1,
        onConfigure: (db) => db.execute('PRAGMA foreign_keys = ON'),
        onCreate: (db, version) async {
          await db.execute('''
            CREATE TABLE mata_kuliah (
              id INTEGER PRIMARY KEY AUTOINCREMENT,
              nama TEXT NOT NULL,
              sks INTEGER NOT NULL
            )
```

```
          ''');
          await db.execute('''
            CREATE TABLE tugas (
              id INTEGER PRIMARY KEY AUTOINCREMENT,
              judul TEXT NOT NULL,
              matkul_id INTEGER NOT NULL,
              FOREIGN KEY (matkul_id) REFERENCES mata_kuliah(id) ON DELETE CASCADE
            )
          ''');
          // Data awal
          final mobile = await db.insert('mata_kuliah', {'nama': 'Pemrograman Mobile', 'sks': 3});
          final basdat = await db.insert('mata_kuliah', {'nama': 'Basis Data', 'sks': 3});
          await db.insert('tugas', {'judul': 'Laporan Praktikum 10', 'matkul_id': mobile});
          await db.insert('tugas', {'judul': 'Proyek CRUD', 'matkul_id': mobile});
          await db.insert('tugas', {'judul': 'Kuis ERD', 'matkul_id': basdat});
        },
      );

  /// Mata kuliah + jumlah tugasnya (LEFT JOIN agar matkul tanpa tugas tetap muncul)
  static Future<List<MataKuliah>> semuaMatkul() async {
    final db = await database;
    final rows = await db.rawQuery('''
      SELECT m.id, m.nama, m.sks, COUNT(t.id) AS jumlah_tugas
      FROM mata_kuliah m
      LEFT JOIN tugas t ON t.matkul_id = m.id
      GROUP BY m.id
      ORDER BY m.nama
    ''');
    return rows.map(MataKuliah.fromMap).toList();
  }

  /// Semua tugas beserta nama mata kuliahnya (INNER JOIN)
  static Future<List<TugasDenganMatkul>> semuaTugas() async {
    final db = await database;
    final rows = await db.rawQuery('''
      SELECT t.id, t.judul, m.nama AS matkul_nama
      FROM tugas t
      JOIN mata_kuliah m ON m.id = t.matkul_id
      ORDER BY m.nama, t.id
    ''');
    return rows.map(TugasDenganMatkul.fromMap).toList();
  }

  static Future<void> tambahTugas(String judul, int matkulId) async {
    final db = await database;
    await db.insert('tugas', {'judul': judul, 'matkul_id': matkulId});
  }

  /// Menghapus matkul: tugas-tugasnya ikut terhapus berkat ON DELETE CASCADE
  static Future<void> hapusMatkul(int id) async {
    final db = await database;
    await db.delete('mata_kuliah', where: 'id = ?', whereArgs: [id]);
  }
}
```

Contoh Lengkap: Mata Kuliah dan Tugasnya

`lib/main.dart`

```dart
// Pastikan sudah: flutter pub add sqflite path, serta membuat
// lib/models/relasi_models.dart dan lib/data/relasi_database.dart
import 'package:flutter/material.dart';

import 'data/relasi_database.dart';
import 'models/relasi_models.dart';

void main() => runApp(const MaterialApp(home: RelasiScreen()));

class RelasiScreen extends StatefulWidget {
  const RelasiScreen({super.key});

  @override
  State<RelasiScreen> createState() => _RelasiScreenState();
}

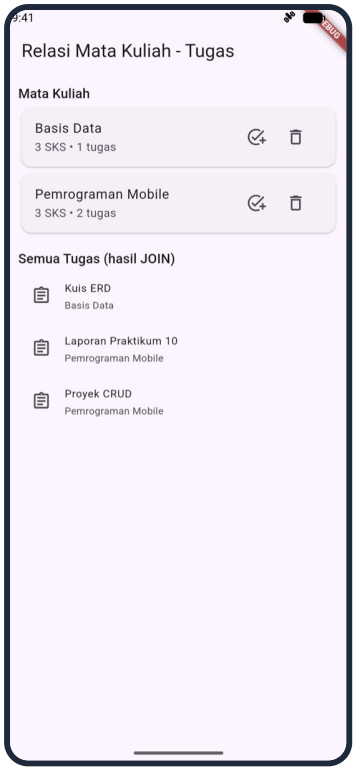
class _RelasiScreenState extends State<RelasiScreen> {
  List<MataKuliah> _matkul = [];
  List<TugasDenganMatkul> _tugas = [];

  @override
  void initState() {
    super.initState();
    _muat();
  }

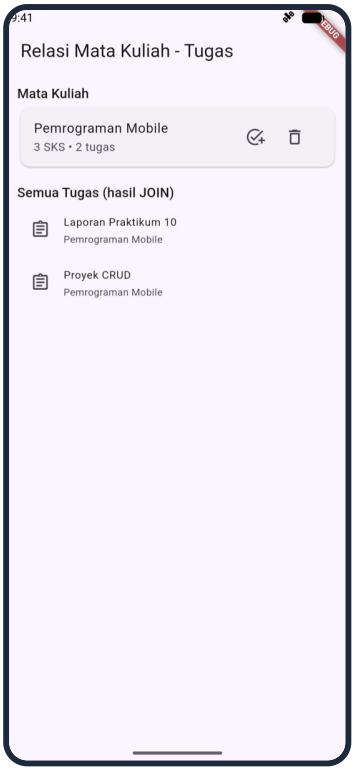
  Future<void> _muat() async {
    final matkul = await RelasiDatabase.semuaMatkul();
    final tugas = await RelasiDatabase.semuaTugas();
    if (!mounted) return;
    setState(() {
      _matkul = matkul;
      _tugas = tugas;
    });
  }

  @override
  Widget build(BuildContext context) {
    final judulStyle = Theme.of(context).textTheme.titleMedium;
    return Scaffold(
      appBar: AppBar(title: const Text('Relasi Mata Kuliah - Tugas')),
      body: ListView(
        padding: const EdgeInsets.all(12),
        children: [
          Text('Mata Kuliah', style: judulStyle),
          for (final m in _matkul)
            Card(
              child: ListTile(
                title: Text(m.nama),
                subtitle: Text('${m.sks} SKS • ${m.jumlahTugas} tugas'),
                trailing: Row(
                  mainAxisSize: MainAxisSize.min,
                  children: [
                    IconButton(
```

```
                    tooltip: 'Tambah tugas',
                    icon: const Icon(Icons.add_task),
                    onPressed: () async {
                      await RelasiDatabase.tambahTugas('Tugas baru ${m.nama}', m.id!);
                      await _muat();
                    },
                  ),
                  IconButton(
                    tooltip: 'Hapus matkul',
                    icon: const Icon(Icons.delete_outline),
                    onPressed: () async {
                      await RelasiDatabase.hapusMatkul(m.id!);
                      await _muat();
                    },
                  ),
                ],
              ),
            ),
          ),
        ),
        const SizedBox(height: 16),
        Text('Semua Tugas (hasil JOIN)', style: judulStyle),
        for (final t in _tugas)
          ListTile(
            dense: true,
            leading: const Icon(Icons.assignment_outlined),
            title: Text(t.judul),
            subtitle: Text(t.matkulNama),
          ),
      ],
    ),
  );
  }
}
```



Data awal: matkul dan hasil JOIN



Basis Data dihapus: tugas Kuis ERD ikut hilang (CASCADE)

Penjelasan: tekan ikon hapus pada "Basis Data". Matkul hilang **dan** tugas "Kuis ERD" juga hilang dari daftar JOIN, padahal kita hanya menjalankan satu `DELETE` pada tabel `mata_kuliah`. Itulah efek `ON DELETE CASCADE`.

⚠️ Kesalahan Umum

- Lupa `PRAGMA foreign_keys = ON`: CASCADE tidak berjalan dan tugas "yatim" (menunjuk matkul yang sudah tidak ada) tetap tersimpan.
- Memakai `JOIN` biasa saat ingin menampilkan matkul yang belum punya tugas. Gunakan `LEFT JOIN`.
- Dua tabel punya kolom bernama sama (`id`, `nama`). Gunakan alias (`m.nama AS matkul_nama`) agar kunci Map tidak bertabrakan.
- Melakukan query per item di dalam loop (N+1 query). Satu query JOIN jauh lebih efisien.

12. Studi Kasus: Aplikasi Catatan Kuliah Multi-File

Kita gabungkan semuanya menjadi aplikasi **Catatan Kuliah** yang sepenuhnya offline: tambah, edit, hapus (dengan urungkan), sematkan (pin), cari, dan filter kategori. Arsitekturnya sama seperti Modul 09, hanya sumber datanya SQLite: **Model → Database → Provider → Screen**.

Struktur Folder

```
lib/
├── main.dart
├── models/
|   └── catatan.dart
├── data/
|   └── catatan_database.dart
├── providers/
|   └── catatan_provider.dart
└── screens/
    ├── catatan_list_screen.dart
    └── catatan_form_screen.dart
```

```
flutter pub add sqflite path provider
```

1. Model

`lib/models/catatan.dart`

```dart
class Catatan {
  static const kategoriList = ['Kuliah', 'Tugas', 'Organisasi', 'Pribadi'];

  final int? id;
  final String judul;
  final String isi;
  final String kategori;
  final bool disematkan;
  final DateTime diubah;

  const Catatan({
    this.id,
    required this.judul,
    required this.isi,
    this.kategori = 'Kuliah',
    this.disematkan = false,
    required this.diubah,
  });

  Map<String, Object?> toMap() => {
        if (id != null) 'id': id,
        'judul': judul,
        'isi': isi,
        'kategori': kategori,
        'disematkan': disematkan ? 1 : 0,
        'diubah': diubah.toIso8601String(),
```

```
      };

  factory Catatan.fromMap(Map<String, Object?> m) => Catatan(
        id: m['id'] as int?,
        judul: m['judul'] as String,
        isi: m['isi'] as String,
        kategori: m['kategori'] as String,
        disematkan: m['disematkan'] == 1,
        diubah: DateTime.parse(m['diubah'] as String),
      );

  Catatan copyWith({String? judul, String? isi, String? kategori, bool? disematkan, DateTime?
diubah}) {
    return Catatan(
      id: id,
      judul: judul ?? this.judul,
      isi: isi ?? this.isi,
      kategori: kategori ?? this.kategori,
      disematkan: disematkan ?? this.disematkan,
      diubah: diubah ?? this.diubah,
    );
  }
}
```

2. Lapisan Database

`lib/data/catatan_database.dart`

```
import 'package:path/path.dart';
import 'package:sqflite/sqflite.dart';

import '../models/catatan.dart';

class CatatanDatabase {
  CatatanDatabase._();
  static final CatatanDatabase instance = CatatanDatabase._();

  static const _tabel = 'catatan';
  Database? _db;

  Future<Database> get _database async => _db ??= await openDatabase(
        join(await getDatabasesPath(), 'catatan_kuliah.db'),
        version: 1,
        onCreate: (db, version) => db.execute('''
          CREATE TABLE $_tabel (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            judul TEXT NOT NULL,
            isi TEXT NOT NULL,
            kategori TEXT NOT NULL,
            disematkan INTEGER NOT NULL DEFAULT 0,
            diubah TEXT NOT NULL
          )
        '''),
      );

  /// Ambil catatan: yang disematkan di atas, lalu yang terbaru.
  Future<List<Catatan>> getAll({String kata = '', String? kategori}) async {
    final db = await _database;
```

```
    final where = <String>[];
    final args = <Object?>[];
    if (kata.isNotEmpty) {
      where.add('(judul LIKE ? OR isi LIKE ?)');
      args.addAll(['%$kata%', '%$kata%']);
    }
    if (kategori != null) {
      where.add('kategori = ?');
      args.add(kategori);
    }
    final rows = await db.query(
      _tabel,
      where: where.isEmpty ? null : where.join(' AND '),
      whereArgs: args.isEmpty ? null : args,
      orderBy: 'disematkan DESC, diubah DESC',
    );
    return rows.map(Catatan.fromMap).toList();
  }

  Future<int> insert(Catatan c) async => (await _database).insert(_tabel, c.toMap());

  Future<void> update(Catatan c) async {
    final db = await _database;
    await db.update(_tabel, c.toMap(), where: 'id = ?', whereArgs: [c.id]);
  }

  Future<void> delete(int id) async {
    final db = await _database;
    await db.delete(_tabel, where: 'id = ?', whereArgs: [id]);
  }
}
```

3. Provider

lib/providers/catatan_provider.dart

```
import 'package:flutter/foundation.dart';

import '../data/catatan_database.dart';
import '../models/catatan.dart';

class CatatanProvider extends ChangeNotifier {
  final CatatanDatabase _db;
  CatatanProvider(this._db);

  List<Catatan> _items = [];
  bool _loading = true;
  String _kata = '';
  String? _kategori;

  List<Catatan> get items => List.unmodifiable(_items);
  bool get loading => _loading;
  String? get kategori => _kategori;
  bool get sedangMemfilter => _kata.isNotEmpty || _kategori != null;

  Future<void> muat() async {
    _items = await _db.getAll(kata: _kata, kategori: _kategori);
    _loading = false;
```

```
    notifyListeners();
  }

  Future<void> cari(String kata) {
    _kata = kata.trim();
    return muat();
  }

  Future<void> pilihKategori(String? kategori) {
    _kategori = kategori;
    return muat();
  }

  Future<void> simpan(Catatan c) async {
    final baru = c.copyWith(diubah: DateTime.now());
    if (c.id == null) {
      await _db.insert(baru);
    } else {
      await _db.update(baru);
    }
    await muat();
  }

  Future<void> toggleSemat(Catatan c) async {
    await _db.update(c.copyWith(disematkan: !c.disematkan));
    await muat();
  }

  Future<void> hapus(Catatan c) async {
    _items.remove(c); // hilang dari layar seketika (dibutuhkan Dismissible)
    notifyListeners();
    await _db.delete(c.id!);
  }

  /// Urungkan hapus: masukkan lagi lengkap dengan id lamanya
  Future<void> pulihkan(Catatan c) async {
    await _db.insert(c);
    await muat();
  }
}
```

4. Layar Daftar

lib/screens/catatan_list_screen.dart

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/catatan.dart';
import '../providers/catatan_provider.dart';
import 'catatan_form_screen.dart';

class CatatanListScreen extends StatelessWidget {
  const CatatanListScreen({super.key});

  void _bukaForm(BuildContext context, [Catatan? c]) {
    Navigator.push(context, MaterialPageRoute(builder: (_) => CatatanFormScreen(catatan: c)));
  }
```

```dart
@override
Widget build(BuildContext context) {
  final prov = context.watch<CatatanProvider>();

  return Scaffold(
    appBar: AppBar(title: const Text('Catatan Kuliah')),
    body: Column(
      children: [
        Padding(
          padding: const EdgeInsets.fromLTRB(12, 12, 12, 4),
          child: SearchBar(
            hintText: 'Cari judul atau isi catatan',
            leading: const Icon(Icons.search),
            onChanged: context.read<CatatanProvider>().cari,
          ),
        ),
        SizedBox(
          height: 48,
          child: ListView(
            scrollDirection: Axis.horizontal,
            padding: const EdgeInsets.symmetric(horizontal: 12),
            children: [
              ChoiceChip(
                label: const Text('Semua'),
                selected: prov.kategori == null,
                onSelected: (_) => prov.pilihKategori(null),
              ),
              for (final k in Catatan.kategoriList)
                Padding(
                  padding: const EdgeInsets.only(left: 8),
                  child: ChoiceChip(
                    label: Text(k),
                    selected: prov.kategori == k,
                    onSelected: (v) => prov.pilihKategori(v ? k : null),
                  ),
                ),
            ],
          ),
        ),
        Expanded(
          child: prov.loading
              ? const Center(child: CircularProgressIndicator())
              : prov.items.isEmpty
                  ? Center(
                      child: Text(prov.sedangMemfilter
                          ? 'Tidak ada catatan yang cocok.'
                          : 'Belum ada catatan. Tekan + untuk menulis.'),
                    )
                  : ListView.builder(
                      padding: const EdgeInsets.only(bottom: 80),
                      itemCount: prov.items.length,
                      itemBuilder: (context, i) => _CatatanTile(
                        catatan: prov.items[i],
                        onTap: () => _bukaForm(context, prov.items[i]),
                      ),
                    ),
        ),
      ],
```

```
      ),
      floatingActionButton: FloatingActionButton.extended(
        onPressed: () => _bukaForm(context),
        icon: const Icon(Icons.edit_note),
        label: const Text('Tulis'),
      ),
    );
  }
}

class _CatatanTile extends StatelessWidget {
  final Catatan catatan;
  final VoidCallback onTap;
  const _CatatanTile({required this.catatan, required this.onTap});

  @override
  Widget build(BuildContext context) {
    final prov = context.read<CatatanProvider>();
    final d = catatan.diubah;
    return Dismissible(
      key: ValueKey(catatan.id),
      direction: DismissDirection.endToStart,
      background: Container(
        color: Colors.red,
        alignment: Alignment.centerRight,
        padding: const EdgeInsets.only(right: 20),
        child: const Icon(Icons.delete, color: Colors.white),
      ),
      onDismissed: (_) {
        final messenger = ScaffoldMessenger.of(context);
        prov.hapus(catatan);
        messenger.showSnackBar(SnackBar(
          content: Text('"${catatan.judul}" dihapus'),
          action: SnackBarAction(label: 'URUNGKAN', onPressed: () => prov.pulihkan(catatan)),
        ));
      },
      child: Card(
        margin: const EdgeInsets.symmetric(horizontal: 12, vertical: 4),
        child: ListTile(
          onTap: onTap,
          title: Text(catatan.judul, maxLines: 1, overflow: TextOverflow.ellipsis),
          subtitle: Text(
            '${catatan.kategori} • ${d.day}/${d.month}/${d.year}\n${catatan.isi}',
            maxLines: 3,
            overflow: TextOverflow.ellipsis,
          ),
          isThreeLine: true,
          trailing: IconButton(
            tooltip: catatan.disematkan ? 'Lepas sematan' : 'Sematkan',
            icon: Icon(catatan.disematkan ? Icons.push_pin : Icons.push_pin_outlined),
            onPressed: () => prov.toggleSemat(catatan),
          ),
        ),
      ),
    );
  }
}
```

5. Layar Form (Tambah & Edit)

`lib/screens/catatan_form_screen.dart`

```
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../models/catatan.dart';
import '../providers/catatan_provider.dart';

class CatatanFormScreen extends StatefulWidget {
  final Catatan? catatan; // null = catatan baru
  const CatatanFormScreen({super.key, this.catatan});

  @override
  State<CatatanFormScreen> createState() => _CatatanFormScreenState();
}

class _CatatanFormScreenState extends State<CatatanFormScreen> {
  final _formKey = GlobalKey<FormState>();
  late final TextEditingController _judul =
      TextEditingController(text: widget.catatan?.judul);
  late final TextEditingController _isi = TextEditingController(text: widget.catatan?.isi);
  late String _kategori = widget.catatan?.kategori ?? Catatan.kategoriList.first;

  @override
  void dispose() {
    _judul.dispose();
    _isi.dispose();
    super.dispose();
  }

  Future<void> _simpan() async {
    if (!_formKey.currentState!.validate()) return;
    final lama = widget.catatan;
    final data = lama == null
        ? Catatan(judul: _judul.text.trim(), isi: _isi.text.trim(), kategori: _kategori, diubah:
DateTime.now())
        : lama.copyWith(judul: _judul.text.trim(), isi: _isi.text.trim(), kategori: _kategori);
    await context.read<CatatanProvider>().simpan(data);
    if (mounted) Navigator.pop(context);
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.catatan == null ? 'Catatan Baru' : 'Edit Catatan'),
        actions: [IconButton(tooltip: 'Simpan', onPressed: _simpan, icon: const Icon(Icons.check))],
      ),
      body: Form(
        key: _formKey,
        child: ListView(
          padding: const EdgeInsets.all(16),
          children: [
            TextFormField(
              controller: _judul,
              decoration: const InputDecoration(labelText: 'Judul', border: OutlineInputBorder()),
```

```
              validator: (v) => (v == null || v.trim().isEmpty) ? 'Judul wajib diisi' : null,
            ),
            const SizedBox(height: 12),
            DropdownButtonFormField<String>(
              initialValue: _kategori,
              decoration: const InputDecoration(labelText: 'Kategori', border:
OutlineInputBorder()),
              items: [
                for (final k in Catatan.kategoriList) DropdownMenuItem(value: k, child: Text(k)),
              ],
              onChanged: (v) => setState(() => _kategori = v ?? _kategori),
            ),
            const SizedBox(height: 12),
            TextFormField(
              controller: _isi,
              minLines: 6,
              maxLines: 12,
              decoration: const InputDecoration(
                labelText: 'Isi catatan',
                alignLabelWithHint: true,
                border: OutlineInputBorder(),
              ),
              validator: (v) => (v == null || v.trim().isEmpty) ? 'Isi wajib diisi' : null,
            ),
            const SizedBox(height: 16),
            FilledButton.icon(onPressed: _simpan, icon: const Icon(Icons.save), label: const
Text('Simpan')),
          ],
        ),
      ),
    );
  }
}
```

6. `main.dart`

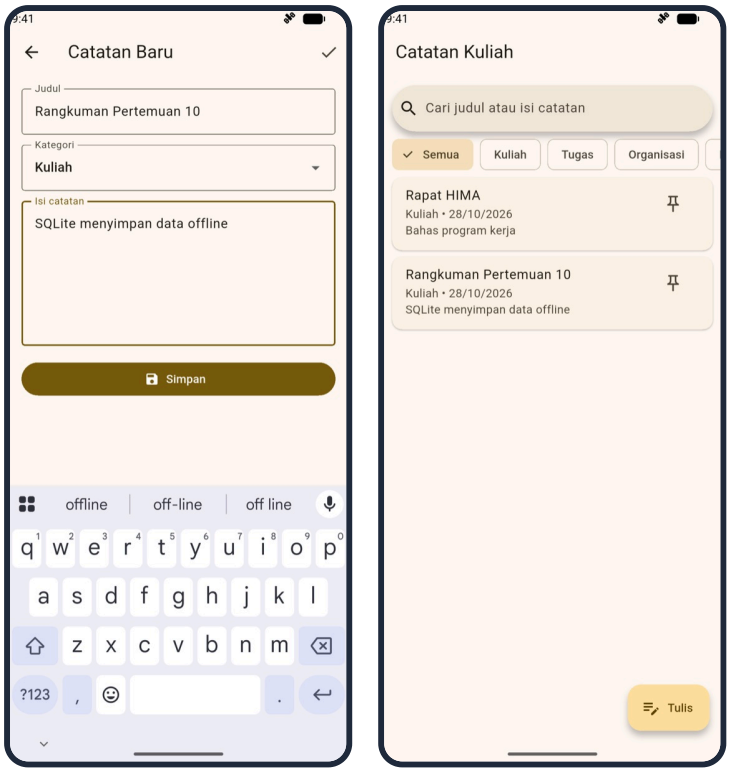
`lib/main.dart`

```
// CATATAN: sqflite hanya berjalan di Android, iOS, dan macOS.
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import 'data/catatan_database.dart';
import 'providers/catatan_provider.dart';
import 'screens/catatan_list_screen.dart';

void main() {
  runApp(
    ChangeNotifierProvider(
      create: (_) => CatatanProvider(CatatanDatabase.instance)..muat(),
      child: MaterialApp(
        title: 'Catatan Kuliah',
        debugShowCheckedModeBanner: false,
        theme: ThemeData(colorSchemeSeed: Colors.amber),
        home: const CatatanListScreen(),
      ),
    ),
```

```
    );
  }
```



Form catatan baru | Dua catatan tersimpan di SQLite

Penjelasan alur:

- **Pencarian & filter** tidak dilakukan di Dart, melainkan diteruskan ke SQL (`LIKE` dan `kategori = ?`) lewat `CatatanDatabase.getAll`.
- **Urutan** `disematkan DESC, diubah DESC` membuat catatan yang di-pin selalu di atas, lalu yang paling baru diubah.
- **Hapus** menghapus dari list lebih dulu (kebutuhan `Dismissible`) lalu dari database. URUNGKAN memasukkan kembali objek beserta id lamanya.
- **Form** memakai `late final` controller yang diinisialisasi dari `widget.catatan`, sehingga satu layar melayani tambah dan edit.

> ⚠️ **Kesalahan Umum**
>
> - Lupa `..muat()` saat membuat provider: spinner tidak pernah hilang karena `_loading` tetap `true`.
> - Memanggil `Navigator.pop` sebelum `await simpan(...)` selesai, sehingga daftar belum ter-update saat kembali.
> - Memakai `value:` pada `DropdownButtonFormField`; versi Flutter terbaru memakai `initialValue:`.

13. Menyimpan File dengan path_provider

Kadang data lebih cocok disimpan sebagai **file**: ekspor catatan ke teks, cache JSON dari API, atau foto. Package `path_provider` memberi tahu lokasi folder yang boleh dipakai aplikasi, lalu kita menulis file dengan `dart:io`.

```
flutter pub add path_provider
```

Folder yang Tersedia

Fungsi	Isi yang cocok	Bisa dihapus sistem?
`getApplicationDocumentsDirectory()`	Data buatan pengguna (ekspor, dokumen)	Tidak
`getApplicationSupportDirectory()`	File internal aplikasi yang tidak dilihat pengguna	Tidak
`getTemporaryDirectory()`	Cache sementara (gambar, respons API)	Ya, kapan saja
`getDatabasesPath()` (sqflite)	File database SQLite	Tidak

Contoh Kecil: Tulis dan Baca File Teks

```dart
import 'dart:io';

import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';

Future<File> _fileCatatan() async {
  final dir = await getApplicationDocumentsDirectory();
  return File(p.join(dir.path, 'catatan.txt'));
}

Future<void> simpanTeks(String isi) async {
  final file = await _fileCatatan();
  await file.writeAsString(isi); // menimpa isi lama
}

Future<String> bacaTeks() async {
  final file = await _fileCatatan();
  if (!await file.exists()) return ''; // belum pernah disimpan
  return file.readAsString();
}
```

Contoh Lengkap: Buku Harian Sederhana

`lib/main.dart`

```dart
// Pastikan sudah: flutter pub add path_provider path
import 'dart:io';

import 'package:flutter/material.dart';
import 'package:path/path.dart' as p;
import 'package:path_provider/path_provider.dart';

void main() => runApp(const MaterialApp(home: BukuHarianScreen()));

class BukuHarianScreen extends StatefulWidget {
  const BukuHarianScreen({super.key});

  @override
  State<BukuHarianScreen> createState() => _BukuHarianScreenState();
}

class _BukuHarianScreenState extends State<BukuHarianScreen> {
  final _ctrl = TextEditingController();
  String _info = 'Memuat...';

  Future<File> get _file async {
    final dir = await getApplicationDocumentsDirectory();
    return File(p.join(dir.path, 'buku_harian.txt'));
  }

  @override
  void initState() {
    super.initState();
    _muat();
  }

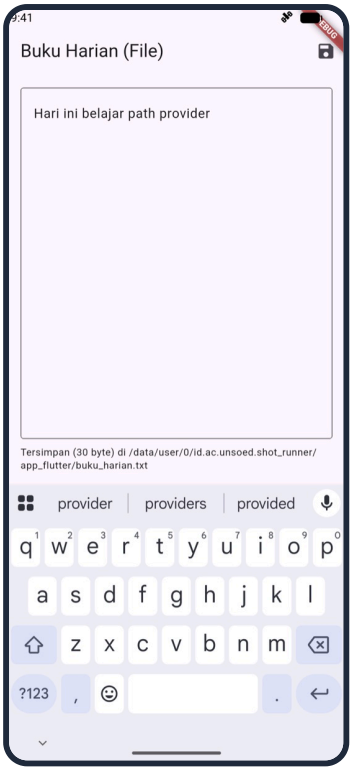
  @override
  void dispose() {
    _ctrl.dispose();
    super.dispose();
  }

  Future<void> _muat() async {
    final file = await _file;
    final ada = await file.exists();
    final isi = ada ? await file.readAsString() : '';
    if (!mounted) return;
    setState(() {
      _ctrl.text = isi;
      _info = ada ? 'Dibaca dari ${file.path}' : 'File belum ada';
    });
  }

  Future<void> _simpan() async {
    final file = await _file;
    await file.writeAsString(_ctrl.text);
    final ukuran = await file.length();
    if (!mounted) return;
    setState(() => _info = 'Tersimpan ($ukuran byte) di ${file.path}');
```

```
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Buku Harian (File)'),
        actions: [IconButton(tooltip: 'Simpan', onPressed: _simpan, icon: const Icon(Icons.save))],
      ),
      body: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          children: [
            Expanded(
              child: TextField(
                controller: _ctrl,
                expands: true,
                maxLines: null,
                textAlignVertical: TextAlignVertical.top,
                decoration: const InputDecoration(
                  hintText: 'Tulis ceritamu hari ini...',
                  border: OutlineInputBorder(),
                ),
              ),
            ),
            const SizedBox(height: 8),
            Text(_info, style: Theme.of(context).textTheme.bodySmall),
          ],
        ),
      ),
    );
  }
}
```



Teks tersimpan ke file di folder dokumen aplikasi

Penjelasan: `File.writeAsString` dan `readAsString` berasal dari `dart:io`. Jalur folder diambil dari `path_provider`, lalu digabung dengan nama file memakai `p.join` agar pemisah folder benar di semua platform.

> ⚠️ **Kesalahan Umum**
>
> - Menulis path secara manual (`'/sdcard/...'`): tidak portabel dan sering ditolak izin. Selalu pakai `path_provider`.
> - Menyimpan data penting di `getTemporaryDirectory()` : bisa dihapus sistem sewaktu-waktu.
> - Membaca file yang belum ada tanpa cek `exists()` : muncul `PathNotFoundException`.
> - `dart:io` tidak tersedia di Flutter Web.

14. Perbandingan: SharedPreferences, SQLite, Hive, Supabase

Setelah mencoba SharedPreferences dan SQLite, mari bandingkan dengan dua teknologi yang akan dipelajari berikutnya: **Hive** (Modul 11, NoSQL key-value lokal) dan **Supabase** (Modul 12, database cloud).

Aspek	SharedPreferences	SQLite (sqflite)	Hive (Modul 11)	Supabase (Modul 12)
Lokasi data	Lokal (HP)	Lokal (HP)	Lokal (HP)	Cloud (server PostgreSQL)
Model data	Key-value sederhana	Tabel relasional (SQL)	Key-value NoSQL berisi objek Dart	Tabel relasional (PostgreSQL) via API
Tipe data	bool, int, double, String, List<String>	INTEGER, REAL, TEXT, BLOB	Tipe Dart + objek kustom (adapter)	Semua tipe PostgreSQL, JSON
Query / filter	Tidak ada (ambil per kunci)	Sangat kuat: WHERE, JOIN, GROUP BY	Filter di Dart (`where`)	Kuat: filter, relasi, RPC
Relasi antar data	-	Ya (foreign key)	Manual	Ya (foreign key)
Perlu internet	Tidak	Tidak	Tidak	Ya (untuk sinkron)
Berbagi antar perangkat/pengguna	Tidak	Tidak	Tidak	Ya, plus autentikasi
Dukungan Web	Ya	Tidak (sqflite)	Ya	Ya
Kurva belajar	Sangat mudah	Sedang (perlu SQL & migrasi)	Mudah	Sedang (perlu setup proyek)
Contoh ideal	Tema, onboarding, bahasa	Catatan, transaksi, KRS offline	Cache data, keranjang belanja, favorit	Aplikasi multi-user, login, data bersama

Kombinasi yang Umum di Aplikasi Nyata

- **SharedPreferences** untuk pengaturan + **SQLite/Hive** untuk data offline.
- **Supabase** sebagai sumber data utama + **Hive/SQLite** sebagai cache agar aplikasi tetap bisa dibuka tanpa internet (*offline-first*).

> ℹ️ **Selanjutnya**
>
> **Modul 11 (Hive)** menunjukkan cara menyimpan objek Dart secara langsung tanpa menulis SQL, termasuk di Flutter Web. **Modul 12 (Supabase)** memindahkan data ke cloud sehingga bisa diakses banyak perangkat dan dilengkapi login. Pola Repository yang dipelajari di Modul 09 dan 10 membuat perpindahan ini mudah: cukup ganti implementasi lapisan data.

Tugas Praktikum: Praktikum 10: Aplikasi Tugas Kuliah Offline dengan SQLite & SharedPreferences

Sasaran Praktikum

Membangun aplikasi manajemen tugas kuliah yang sepenuhnya berfungsi tanpa internet, menyimpan pengaturan di SharedPreferences dan data berelasi di SQLite.

Langkah Pengerjaan

1. Buat tabel `mata_kuliah` (id, nama, sks) dan `tugas` (id, judul, matkul_id, tenggat, isSelesai) dengan foreign key `ON DELETE CASCADE` dan `PRAGMA foreign_keys = ON`.
2. Buat model `MataKuliah` dan `Tugas` lengkap dengan `toMap`, `fromMap`, dan `copyWith` (konversi bool dan DateTime).
3. Buat kelas database singleton dengan operasi CRUD bertipe; semua filter wajib memakai `whereArgs`.
4. Tampilkan daftar tugas (hasil JOIN dengan nama mata kuliah) memakai Provider; tambah dan edit melalui satu form dengan validasi dan pemilih tanggal.
5. Centang untuk menandai selesai (UPDATE) dan geser untuk menghapus (DELETE) dengan tombol URUNGKAN.
6. Tambahkan pencarian judul (LIKE) dan filter per mata kuliah, serta ringkasan jumlah tugas per mata kuliah (GROUP BY).
7. Naikkan versi database menjadi 2 dengan menambah kolom `prioritas` melalui `onUpgrade`, lalu buktikan data lama tidak hilang.
8. Simpan pilihan tema (terang/gelap/sistem) dan urutan daftar terakhir di SharedPreferences; tutup aplikasi sepenuhnya lalu buka kembali untuk membuktikan semua data dan pengaturan tetap tersimpan.

Output yang Diharapkan

Aplikasi tugas kuliah offline yang persisten: data tugas dan mata kuliah tersimpan di SQLite (termasuk setelah migrasi versi 2), pengaturan tema tersimpan di SharedPreferences, dan semuanya tetap ada setelah aplikasi di-restart.

Rubrik Penilaian

Desain skema, relasi & migrasi (25%), Model dan lapisan database bertipe (20%), Fungsionalitas CRUD + pencarian/filter aman (30%), SharedPreferences untuk tema (10%), UX: validasi, urungkan, pesan kosong (15%).

Kuis Pemahaman

1. Kapan kita sebaiknya memilih SharedPreferences dibandingkan SQLite?

A. Ketika hanya perlu menyimpan data pengaturan sederhana seperti preferensi tema gelap dan token autentikasi

B. Ketika perlu menyimpan data ribuan transaksi keuangan yang saling berelasi

C. Ketika aplikasi membutuhkan query SELECT yang kompleks dengan filter JOIN

D. SharedPreferences tidak pernah digunakan di industri

2. Mengapa pada SQLite tipe data Boolean biasanya disimpan sebagai tipe INTEGER (0 dan 1)?

A. Karena SQLite tidak memiliki tipe data Boolean terpisah secara natif, sehingga angka 0 mewakili false dan 1 mewakili true

B. Agar data lebih cepat terkirim ke internet

C. Karena Flutter melarang penggunaan kata true di file SQL

D. Karena SQLite hanya menerima angka ganjil

3. Menurut dokumentasi shared_preferences terbaru, API mana yang dianjurkan untuk proyek baru?

A. SharedPreferences.getInstance() (API lama)

B. SharedPreferencesAsync atau SharedPreferencesWithCache

C. localStorage dari dart:html

D. File teks di folder temporary

4. Kamu menambah kolom baru di CREATE TABLE pada onCreate, tetapi di HP yang sudah memasang versi lama muncul error 'no such column'. Mengapa?

A. sqflite tidak mendukung kolom baru

B. onCreate hanya berjalan saat database belum ada; perlu menaikkan version dan menulis ALTER TABLE di onUpgrade

C. Harus memakai hot reload

D. Nama kolom harus huruf kapital

5. Mana cara yang AMAN dari SQL injection untuk mencari mahasiswa berdasarkan input pengguna?

A. db.rawQuery("SELECT * FROM mahasiswa WHERE nama = '$input'")

B. db.query('mahasiswa', where: 'nama = ?', whereArgs: [input])

C. db.execute('SELECT ' + input)

D. db.query('mahasiswa', where: "nama = '$input'")

6. Apa yang terjadi jika sebuah exception dilempar di dalam callback db.transaction?

A. Hanya perintah terakhir yang dibatalkan

B. Semua perubahan di dalam transaksi dibatalkan (rollback)

C. Database terhapus

D. Exception diabaikan dan data tetap tersimpan

7. Mengapa ON DELETE CASCADE tidak berjalan padahal sudah ditulis di CREATE TABLE?

A. SQLite tidak mendukung foreign key sama sekali

B. Foreign key di SQLite nonaktif secara default; harus menjalankan PRAGMA foreign_keys = ON (misalnya di onConfigure)

C. Harus memakai rawDelete

D. CASCADE hanya berjalan di iOS

8. Folder mana yang TIDAK cocok untuk menyimpan data penting pengguna karena bisa dihapus sistem kapan saja?

A. getApplicationDocumentsDirectory()

B. getApplicationSupportDirectory()

C. getTemporaryDirectory()

D. getDatabasesPath()

Kunci Jawaban & Pembahasan

1. A — SharedPreferences dirancang khusus untuk menyimpan pasangan kunci-nilai (key-value) berukuran kecil seperti preferensi pengguna, flags, atau token login. SQLite digunakan saat data berstruktur relasional banyak tabel.

2. A — Engine SQLite tidak memiliki tipe data BOOLEAN khusus tersendiri; nilai boolean disimpan sebagai integer 0 (false) dan 1 (true).

3. B — Sejak versi 2.3.0, SharedPreferences (legacy) direncanakan untuk di-deprecate. Kode baru dianjurkan memakai SharedPreferencesAsync (selalu baca dari disk) atau SharedPreferencesWithCache (baca sinkron dari cache).

4. B — Database yang sudah ada tidak menjalankan onCreate lagi. Perubahan skema untuk pengguna lama dilakukan lewat onUpgrade setelah nomor version dinaikkan.

5. B — Dengan placeholder ? dan whereArgs, nilai dikirim terpisah dari perintah SQL sehingga input seperti x' OR '1'='1 diperlakukan sebagai teks biasa.

6. B — Transaksi bersifat atomik: jika callback melempar exception, sqflite melakukan rollback sehingga tidak ada perubahan yang tersimpan.

7. B — SQLite mematikan penegakan foreign key secara default. PRAGMA foreign_keys = ON perlu dijalankan setiap kali database dibuka, tempat yang tepat adalah onConfigure.

8. C — Folder temporary ditujukan untuk cache; sistem operasi boleh mengosongkannya saat ruang penyimpanan menipis.